Project: STOCK & INVENTORY MANAGEMENT SYSTEM

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		**25%**	**50%**	**75%**	**100%**	**10**
Clarity	**2**					
Content Quality	**4**					
Spelling & Grammar	**2**					
Organization and Formatting	**2**					
					Total obtained mark	
Comments						

Semester: Summer 2025 **Group No:** 02

Name	ID
Md Amir Hamja	242-35-488
Nafisha Anjum	242-35-188
Sabbir Wahed Maruf	242-35-562

Batch: 43th **Section: K1**

Course Name: Software Development Capstone Project

Course Code: SE133

Course Teacher Name: Md. Sakib Ali Mazumder

Designation: Lecturer

Submission Date: 20 August 2025

Stock and Inventory Management System

TABLE OF CONTENT

NUMBER	CONTENT	PAGE NO.
1.	TABLE OF CONTENT	02
2.	DECLARATION BY THE STUDENT	03
3.	ACKNOWLEDGEMENT	04
4.	LOGO	04
5.	ABSTRACT	05
6.	INTRODUCTION	06
7.	LITERATURE REVIEW	06-07
8.	REQUIREMENT	08
9.	USE CASE & DESCRIPTION	09-17
10.	SEQUENCE DIAGRAM	18
11.	ACTIVITY DIAGRAM	19
12.	METHODOLOGY	20
13.	PROJECT PLAN	21
14.	TIME GANTT CHART	21
15.	BUDGET	22
16.	EVALUATION	23
17.	C IMPLEMENTATION	24-28
18.	LIMITATIONS	29-31
19.	REPORT SUMMARY	32-33
20.	SOURCE CODE	33-80
21.	USER MANUAL	81-103
22.	FUTURE SCOPE	103-108
23.	IMPLEMENTATION ROADMAP	108
24.	TECH RECOMMENDATIONS	108
25.	REFERENCE	109
26.	CONCLUSION	109

DECLARATION BY THE STUDENT

We, MD. AMIR HAMJA (242-35-488), NAFISHA ANJUM (242-35- 188), SABBIR WAHED MARUF (242-35-562), hereby declare the project "**Stock and Inventory Management System**", carried out under the guidance of **Md. Sakib Ali Mazumder sir.** This is a record of trustworthy work carried out by us and the result produced by us have not been reproduced/copied from any source.

The result embodied in this project report have not been submitted top any other university. The project proposal is submitted by the student of DAFFODIL INTERNATIONAL UNIVERSITY of the Department of Software Engineering.

MD. AMIR HAMJA	REG : 242-35-488
NAFISHA ANJUM	REG : 242-35-188
SABBIR WAHED MARUF	REG : 242-35-562

Under the esteemed guidance of

MD. SAKIB ALI MAZUMDER

LECTURER OF SOFTWARE ENGINEERING,

DAFFODIL INTERNATIONAL UNIVERSITY

ACKNOWLEDGEMENT

We extend our heartfelt thanks to **Daffodil International University** for providing us with this remarkable opportunity and supporting the development of this project. I would also like to express our appreciation to the **Dept. of Software Engineering**. Particularly our esteemed department head, **Dr. Imran Mahmud**, and our dedicated teachers for their exceptional instruction and commitment to excellence.

For that we take the opportunity to thank all who have rendered their full support to our work. The pleasure, the achievement, the glory, the satisfaction, the reward and the appreciation of our project cannot be thought without a few, who apart from their regular schedule spared their valuable time for us. We thank our guide **Md. Sakib Ali Mazumder sir** for giving us the opportunity to do this project work and her constant help and guidance.

LOGO



ABSTRACT

The Stock and Inventory Management System represents a comprehensive solution designed to address the critical challenges faced by businesses in managing their inventory operations. This project develops a robust, console-based application using the C programming language that facilitates efficient tracking, monitoring, and management of stock levels, customer orders, supplier relationships, and financial transactions. The system incorporates a multi-user architecture supporting three distinct user roles: administrators with full system privileges, employees with operational access, and customers with purchasing capabilities. Through implementation of advanced data structures, file handling mechanisms, and secure authentication protocols, the system ensures data integrity and operational reliability.
The application features real-time stock monitoring with automated low-stock alerts, comprehensive reporting capabilities, and seamless transaction processing.

By utilizing structured programming principles and modular design patterns, the system demonstrates scalability potential while maintaining code maintainability. The file-based data persistence ensures data continuity across sessions, while the ASCII art interface enhances user engagement. This project successfully bridges the gap between theoretical computer science concepts and practical business applications, providing a foundation for future enhancements including database integration, web-based interfaces, and advanced analytics capabilities.

INTRODUCTION

Background: Businesses often struggle with inventory shortages, overstocking, and manual record-keeping. A digital stock management solution can automate and streamline these tasks.

Problem Statement: Manual stock tracking leads to frequent errors, time delays, and poor decision-making. There's a need for a centralized system that manages stock updates, monitors product levels, and handles purchases and sales efficiently.

Objectives:

i. To build a user-friendly platform for managing stock and inventory.

ii. To track products, suppliers, and customers in real-time.

iii. To generate alerts and reports based on inventory thresholds.

iv.

LITERATURE REVIEW

Related Work: Inventory management systems are critical for businesses that handle physical products. Existing systems - such as **Zoho Inventory**, **Odoo**, **Tally ERP**, and **QuickBooks** - offer various features including product tracking, barcode scanning, real-time alerts, and stock-level automation. These systems are widely adopted in retail, warehousing, and manufacturing sectors for:

- Real-time tracking of goods
- Generating reports on inventory status
- Automating reordering processes
- Integrating with sales, purchase, and accounting modules

Research and case studies show that integrating inventory systems with cloud platforms and mobile applications enhances accessibility and operational efficiency. Moreover, the use of **IoT-based sensors** and **RFID** technology in advanced systems has further improved tracking accuracy and stock visibility.

Gap Analysis: Despite these advancements, several gaps still exist in traditional or basic inventory systems, especially in the context of small to medium-sized enterprises (SMEs):

1. **Cost & Complexity**: Most commercial systems are either expensive or overly complex for SMEs, making adoption difficult.
2. **Customization Limitations**: Many off-the-shelf solutions lack flexibility in adapting to specific business needs (custom product categories, dynamic pricing models).
3. **Real-Time Processing**: Delays in reflecting stock changes during peak business hours can cause over-selling or understocking.
4. **Integration Challenges**: Difficulty integrating with third-party tools like accounting systems or supplier databases.
5. **Offline Functionality**: Most modern systems are cloud-based and do not function well without a stable internet connection, which is a limitation in rural or less-connected areas.
6. **Data Visualization**: Basic systems lack intuitive dashboards or visual analytics for decision-makers.

REQUIREMENT

Functional Requirements:

1. **User Authentication:** Secure login for admin and employees.
2. **Product Management:** Add, update, and delete product info.
3. **Stock Monitoring:** Track quantity in/out and auto-update levels.
4. **Supplier Management:** Add/manage supplier data.
5. **Customer Management:** Track customer orders and invoices.
6. **Sales & Purchase Records:** Handle transactions and generate receipts.
7. **Stock Alerts:** Notify when stock is below reorder level.
8. **Reports:** Daily/weekly/monthly sales and stock reports.
9. **Log out:** Secure session termination.

Non-Functional Requirements:

- **Security:** Encrypted login and secure database.
- **Usability:** Simple and intuitive UI/UX.
- **Performance:** Fast response under multiple concurrent transactions.
- **Reliability:** System should be available 24/7.
- **Backup & Recovery:** Periodic backup and restore functionality.

USE CASE

I. Registration/Login

II. Add Product

III. Update Product

IV. Remove Product

V. Product List

VI. Generate Report

VII. Supply Product

VIII. Receive Stock

IX. Stock Alerts

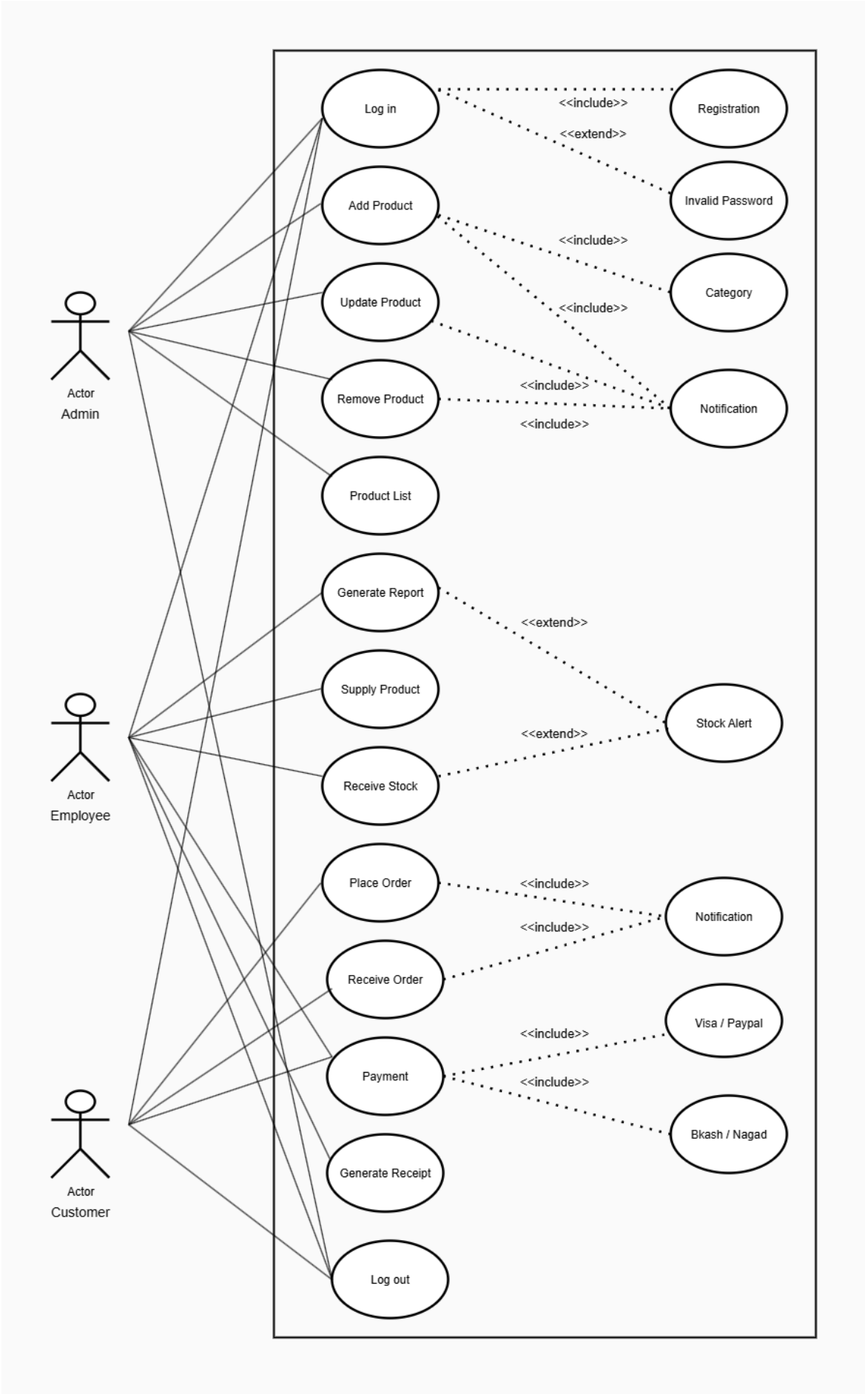
X. Place Order

XI. Receive Order

XII. Payment

XIII. Generate Receipt

XIV. Logout


Actor
Admin
Actor
Employee
Actor
Customer
Log in
Add Product
Update Product
Remove Product
Product List
Generate Report
Supply Product
Receive Stock
Place Order
Receive Order
Payment
Generate Receipt
Log out
Registration
Invalid Password
Category
Notification
Stock Alert
Notification
Visa / Paypal
Bkash / Nagad
<<include>>
<<extend>>
<<include>>
<<include>>
<<include>>
<<include>>
<<extend>>
<<extend>>
<<include>>
<<include>>
<<include>>
<<include>>

USE CASE DESCRIPTION

Use Case: Registration/Login

Use Case	Registration/Login
Goal	Allow users to register and log in to the system securely.
Precondition	User is not logged in.
Success End Con.	User successfully logged in.
Failed End Con.	Invalid credentials or registration error.
Primary Actor	User
Secondary Actor	Admin
Trigger	User accesses the login or registration page.
Description/Main Success Scenario	1. User opens login page. 2. Enters username and password. 3. System verifies credentials. 4. Grants access on success.
Alternative Flows	1. If credentials are incorrect, system shows error. 2. If user is not registered, prompts for registration.
Quality Requirements	1 Secure password handling. 2 Fast login response.

Use Case: Manage User

Use Case	Manage Users
Goal	Allow admin to view, add, update, or delete users.
Precondition	Admin is logged in.
Success End Con.	User information managed successfully.
Failed End Con.	Invalid input or access denied.

Primary Actor	Admin
Secondary Actor	None
Trigger	Admin selects manage users option.
Description/Main Success Scenario	1. Admin selects 'Manage Users'. 2. Chooses to add, update, or delete user. 3. System processes the request. 4. Updates user records.
Alternative Flows	1. If user not found, system informs admin. 2. If input is invalid, prompt to retry.
Quality Requirements	1. Ensure unique user IDs. 2. Secure access control. 3. Accurate data update.

Use Case: Add, Update, Remove Product

Use Case	Add, Update, Remove Product
Goal	Allow admin to manage products in inventory.
Precondition	Admin is logged in.
Success End Con.	Product list updated successfully.
Failed End Con.	Invalid input or product not found.
Primary Actor	Admin
Secondary Actor	None
Trigger	Admin selects product management option.
Description/Main Success Scenario	1. Admin selects add/update/remove. 2. Enters product details or ID. 3. System performs the action. 4. Confirms result.

Alternative Flows	1. Invalid input prompts retry. 2. Nonexistent ID gives error.
Quality Requirements	1. Validate all fields. 2. Prevent duplicate IDs. 3. Save data correctly.

Use Case: Product List

Use Case	Product List
Goal	Display all current products with details.
Precondition	Products exist in inventory.
Success End Con.	Product list is displayed.
Failed End Con.	No products found.
Primary Actor	User, Admin
Secondary Actor	None
Trigger	User selects 'View Product List'.
Description/Main Success Scenario	1. System opens product file. 2. Reads all products. 3. Displays in list format.
Alternative Flows	Empty inventory shows message.
Quality Requirements	1. Fast loading. 2. Clear layout. 3. Accurate data.

Use Case: Generate Report

Use Case	Generate Reports
Goal	Allow admin to generate stock and sales reports.
Precondition	Inventory data exists.
Success End Con.	Report displayed with totals.
Failed End Con.	No data found.
Primary Actor	Admin
Secondary Actor	None

Trigger	Admin selects report option.
Description/Main Success Scenario	1. System gathers all inventory/sales data. 2. Calculates totals. 3. Shows summary report.
Alternative Flows	If file missing, show error.
Quality Requirements	1. Accurate calculations. 2. Quick generation.

Use Case: Supply Products

Use Case	Supply Products
Goal	Allow admin to record supply of new products.
Precondition	Supplier provides product info.
Success End Con.	Supplied stock added.
Failed End Con.	Product or supplier error.
Primary Actor	Admin
Secondary Actor	Supplier
Trigger	Admin selects 'Supply Product'.
Description/Main Success Scenario	1. Admin selects product to supply. 2. Enters supply quantity. 3. System updates stock.
Alternative Flows	Invalid supply quantity prompts error.
Quality Requirements	1. Real-time stock update. 2. Supply history tracking.

Use Case: Receive Stock

Use Case	Receive Stock
Goal	Track received stock into inventory.
Precondition	Stock has been delivered.
Success End Con.	Stock quantity updated.

Failed End Con.	Mismatch in stock or input error.
Primary Actor	Admin
Secondary Actor	Delivery Agent
Trigger	Admin confirms stock received.
Description/Main Success Scenario	1. Admin checks delivery. 2. Confirms quantity received. 3. System updates inventory.
Alternative Flows	- Quantity mismatch raises alert.
Quality Requirements	1. Accurate tracking. 2. Easy editing if error.

Use Case: Stock Alerts

Use Case	Stock Alerts
Goal	Notify admin when stock is low.
Precondition	Products exist in inventory.
Success End Con.	Alert shown or sent.
Failed End Con.	System fails to detect low stock.
Primary Actor	System
Secondary Actor	Admin
Trigger	System checks stock levels automatically.
Description/Main Success Scenario	1. System monitors stock. 2. If product quantity < threshold, alert admin.
Alternative Flows	None.
Quality Requirements	1. Timely notification. 2. Adjustable threshold.

Use Case: Take Order

Use Case	Take Order
Goal	Allow users to place orders for products.
Precondition	Products are available in stock.
Success End Con.	Order placed successfully.

Failed End Con.	Stock unavailable.
Primary Actor	User
Secondary Actor	Admin
Trigger	User selects 'Place Order'.
Description/Main Success Scenario	1. User browses products. 2. Selects items. 3. Confirms order. 4. System processes and stores order.
Alternative Flows	If out of stock, show message.
Quality Requirements	1. Must verify stock before confirm. 2. Order data saved.

Use Case: Payment

Use Case	Payment
Goal	Process payment for orders.
Precondition	Order placed and confirmed.
Success End Con.	Payment successful.
Failed End Con.	Payment failed.
Primary Actor	User
Secondary Actor	Admin
Trigger	User selects 'Pay Now'.
Description/Main Success Scenario	1. User selects payment option. 2. Enters payment details. 3. System processes transaction.
Alternative Flows	Invalid details = error message.
Quality Requirements	1. Secure transaction. 2. Confirmation message.

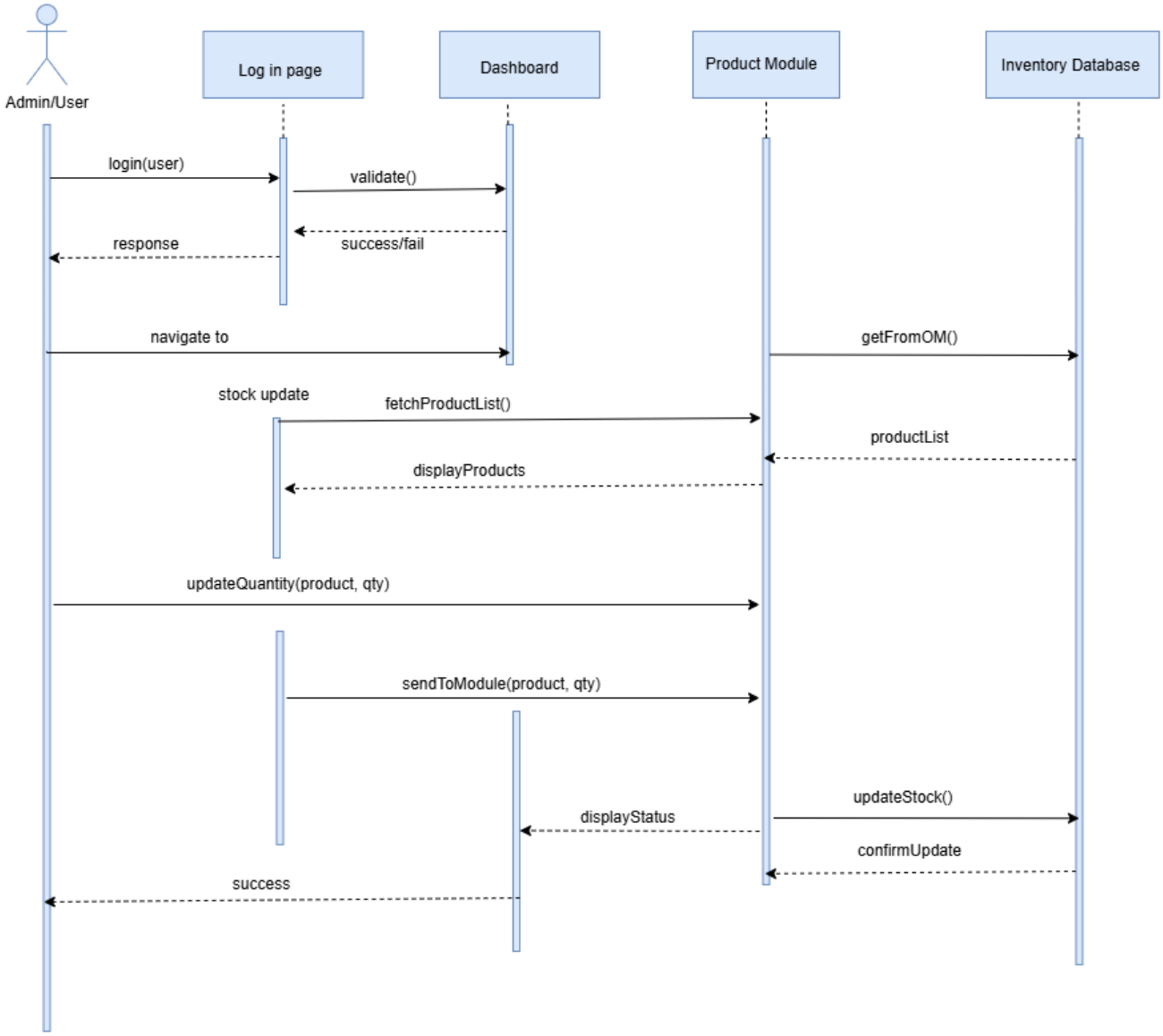
Use Case: Generate Receipts

Use Case	Generate Receipts
Goal	Generate receipt after payment.
Precondition	Payment is successful.
Success End Con.	Receipt displayed or printed.
Failed End Con.	Receipt generation error.
Primary Actor	System
Secondary Actor	User
Trigger	System triggers receipt after payment.
Description/Main Success Scenario	1. System fetches order and payment info. 2. Formats receipt. 3. Displays/prints.
Alternative Flows	System failure shows retry option.
Quality Requirements	1. Clear and complete receipt. 2. Correct total.

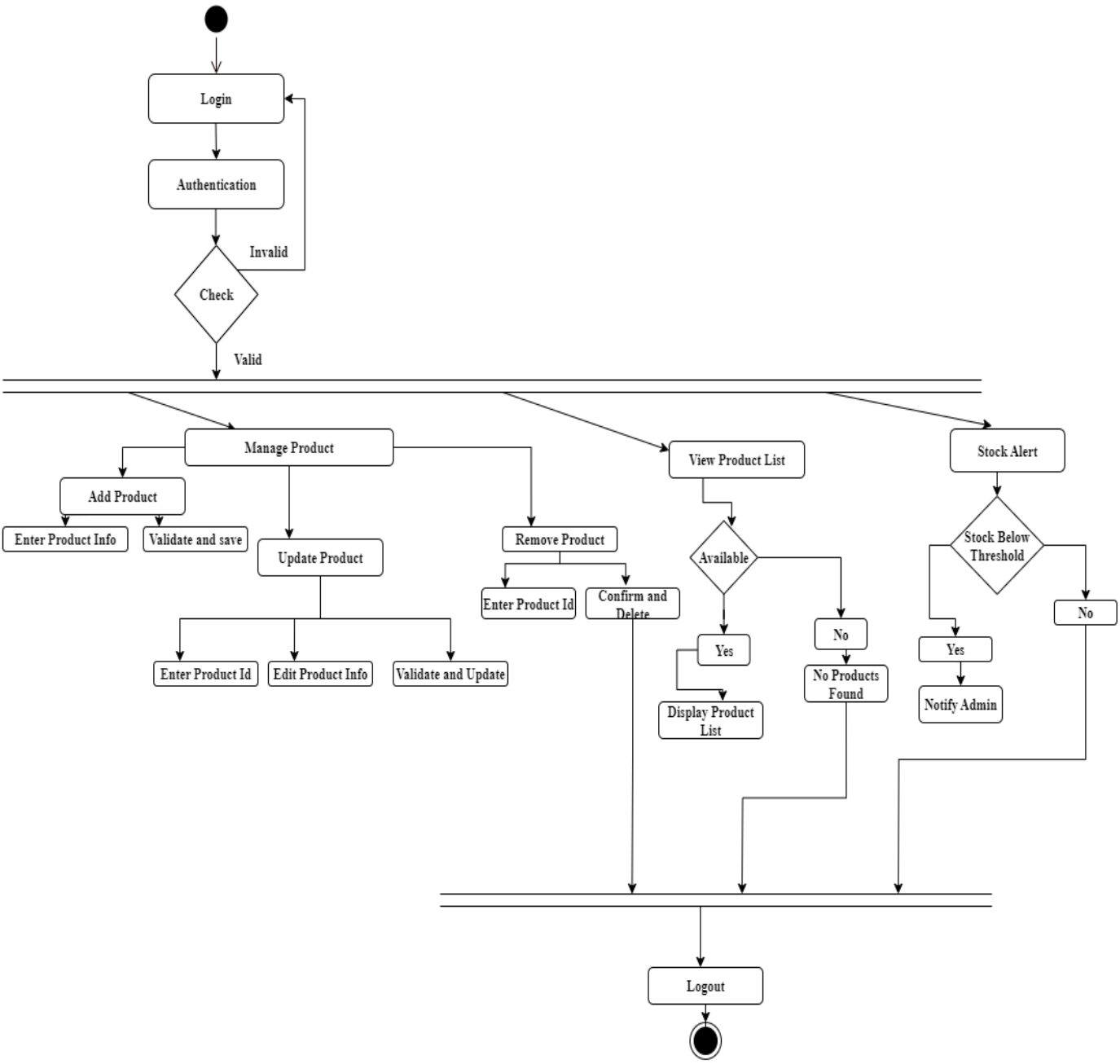
Use Case: Logout

Use Case	Logout
Goal	Safely log out the current user.
Precondition	User is logged in.
Success End Con.	User is logged out.
Failed End Con.	Logout failure.
Primary Actor	User
Secondary Actor	System
Trigger	User clicks 'Logout'.
Description/Main Success Scenario	1. User selects logout. 2. System ends session. 3. Redirects to login page.
Alternative Flows	Session timeout auto logout.
Quality Requirements	1. Session must end fully. 2. Secure redirect.

SEQUENCE DIAGRAM



ACTIVITY DIAGRAM



METHODOLOGY

Approach:

1. Develop a web-based inventory system with role-based access control for admin and employees.
2. Implement real-time stock tracking and alert system for low inventory.
3. Use encryption and secure session management for safe login and data handling.
4. Build a relational database to store product, supplier, customer, and transaction data.

Tools and Techniques:

i. **Software**: Online GBD Compiler & Code-Blocks
ii. **Hardware**: N/A

Project Phases:

i. Requirement Gathering
ii. System Design
iii. Implementation of Core Features (product management, stock in/out, supplier/customer handling)
iv. Testing and Debugging
v. Final Deployment and Documentation.

PROJECT PLAN

Timeline:

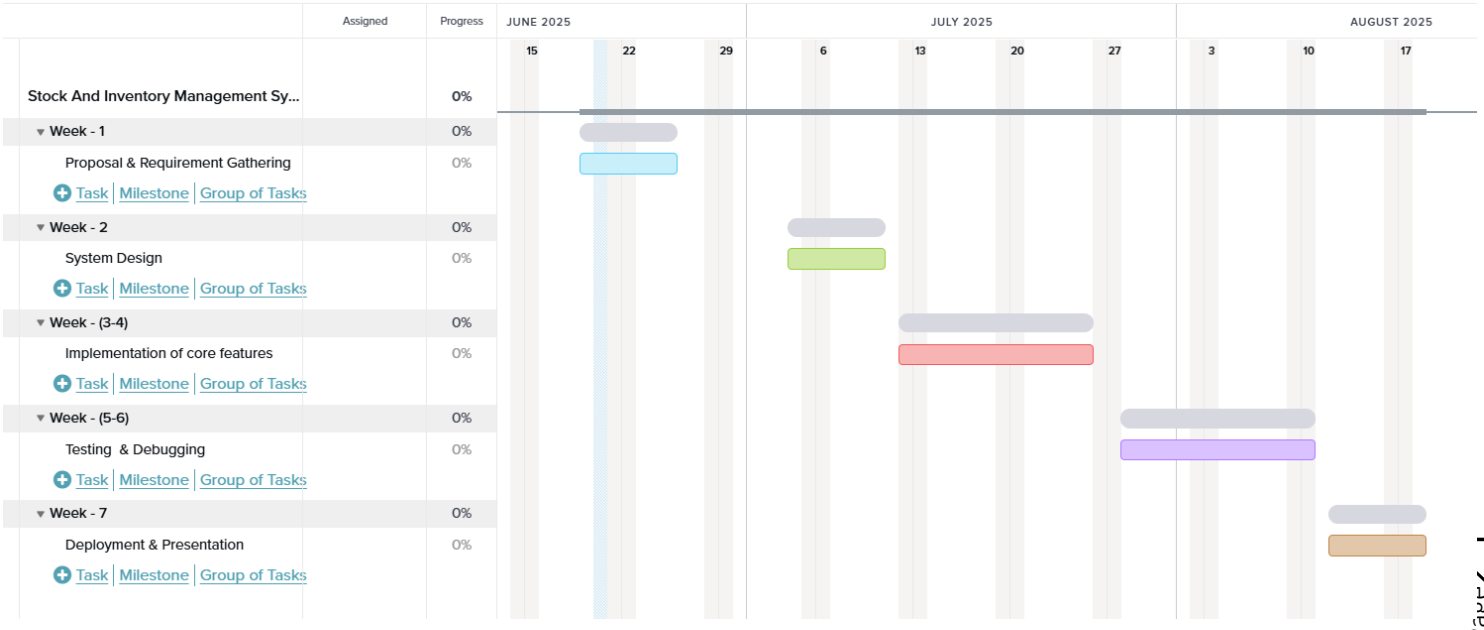
Week 1-2: Project Proposal,Requirements gathering and system design.

Week 3-4: Implementation of core features.

Week 5-6: Testing and debugging.

Week 7 : Final deployment and presentation.

TIME GANTT CHART



BUDGET

Development team cost:

1. **Project manager**: 8000-10000/= BDT
2. **Software develpoers**: 15000-22000/= BDT

Estimated Software Development Costs:

1. 30,000 – 80,000/= BDT depending on the complexity and the size of the team.

Development Tools Cost:

1. **Operating System Licenses** (Windows, Linux): 500 – 5,000/= BDT

2. **Development Tools** : 5,000 – 20,000/= BDT

Hardware Tools Cost:

1. **Standard Development PC/Laptop** : Already Available.

2. **Internet Connection** : Already Available.

EVALUATION

Assessment Criteria:

i. The system must accurately manage **stock-in and stock-out operations** in real-time.

ii. The system must handle **multiple simultaneous transactions** without performance degradation.

iii. The platform must provide a **user-friendly interface** with easy navigation for both admin and employees.

iv. The system must maintain **data integrity** even in the event of abrupt closures or errors.

Evaluation Methods:

User Testing: Real users (admin & staff) will test the full workflow from product entry to stock movement.

System Performance: Evaluate response time and load behavior with increasing numbers of transactions.

Functionality Testing: Verify each module - product management, supplier/customer handling, and reporting - works as expected.

Feedback Collection: Gather feedback from users regarding usability, performance, and interface clarity.

C IMPLEMENTATION

1. Data Structures Implementation

1.1 Structure Definitions

```c
// User management structures
typedef struct {
    char username[30];
    char password[30];
} Admin, Employee, Customer;

// Inventory management structures
typedef struct {
    char name[50];
    float price;
    char category[30];
    int quantity;
} Item;

// Order processing structures
typedef struct {
    int orderId;
    char customerName[30];
    char itemName[50];
    int quantity;
    float totalAmount;
    char status[20];
    char date[20];
} Order;
```

1.2 Array Management

- **Static Arrays**: Utilized for storing user accounts, inventory items, orders, and transactions
- **Array Indexing**: Efficient data retrieval using index-based access
- **Dynamic Sizing**: Counter variables tracking current array utilization
- **Memory Optimization**: Predefined maximum sizes preventing overflow

2. File Handling Implementation

2.1 File I/O Operations

```
// Comprehensive file handling for data persistence
void saveItems() {
    FILE *file = fopen("items.txt", "w");
    // Structured data writing with delimited format
}

void loadItems() {
    FILE *file = fopen("items.txt", "r");
    // Data parsing and structure population
}
```

2.2 Data Persistence Strategy

- **Multiple File System**: Separate files for different data types (items, orders, transactions, users)
- **Delimited Format**: Pipe-separated values for data organization
- **Error Handling**: Comprehensive file operation error checking
- **Backup System**: Timestamped backup file generation

3.2 Input Handling

```
void getHiddenPassword(char *password) {
    // Secure password input with character masking
    // Real-time character processing using getch()
}
```

4. Control Structures Implementation

4.1 Loop Structures

- **While Loops**: Main application loop and menu navigation
- **For Loops**: Array iteration and data processing
- **Do-While Loops**: Input validation and retry mechanisms

4.2 Conditional Logic

- **Switch Statements**: Menu selection and option processing
- **Nested If-Else**: Complex validation and authentication logic
- **Boolean Logic**: Status checking and permission validation

5. Function Implementation

5.1 Modular Programming

- **Function Categories**: Authentication, inventory management, order processing, file operations
- **Parameter Passing**: Efficient data sharing between functions
- **Return Values**: Status indication and data retrieval
- **Global Variables**: State management and data sharing

5.2 Advanced Function Features

```
// Console manipulation functions
int getConsoleWidth() {
    // Windows API integration for dynamic formatting
}

void printCentered(const char *text) {
    // Dynamic text centering based on console width
}
```

6. Memory Management

6.1 Static Memory Allocation

- **Array Declarations**: Fixed-size arrays for predictable memory usage
- **Structure Storage**: Efficient memory layout for data structures
- **String Buffers**: Adequate buffer sizes for string operations

6.2 Memory Optimization

- **Buffer Management**: Appropriate buffer sizes for different operations
- **Memory Reuse**: Efficient structure utilization throughout application lifecycle

7. System Integration

7.1 Operating System Interaction

```
#include <windows.h>
// Console screen buffer manipulation
// System command execution for backup operations
```

7.2 Time Functions

```
void getCurrentDate(char *dateStr) {
    time_t t = time(NULL);
    struct tm tm = *localtime(&t);
    sprintf(dateStr, "%02d/%02d/%04d", tm.tm_mday, tm.tm_mon + 1, tm.tm_year + 1900);
}
```

LIMITATIONS

1.Technical Limitations

1.1 Platform Dependencies

- **Windows-Specific Code**: Heavy reliance on Windows API functions (`windows.h`, `conio.h`)
- **Cross-Platform Compatibility**: Limited portability to Unix/Linux systems without modifications.
- **Console Dependencies**: Tied to console-based interface limiting modern UI expectations.

1.2 Data Storage Limitations

- **File-Based Storage**: No database integration limiting data integrity and concurrent access.
- **No Transaction Management**: Lack of ACID properties in data operations.
- **Limited Backup**: Basic backup system without versioning or incremental backups.
- **Data Scalability**: File-based approach not suitable for large datasets.

1.3 Security Limitations

- **Password Storage**: Passwords stored in plain text without encryption.
- **Session Management**: Basic session handling without timeout mechanisms.
- **Access Control**: Limited role-based permissions without granular controls.
- **Data Validation**: Basic input validation without comprehensive sanitization.

1.4 Concurrency Limitations

- **Single-User Access**: No support for multiple simultaneous users.
- **No Locking Mechanism**: Potential data corruption with concurrent file access.
- **Sequential Processing**: Operations processed sequentially without parallel capabilities.

2.Functional Limitations

2.1 Inventory Management

- **No Barcode Support**: Manual product identification without automation.
- **Limited Categorization**: Fixed category system without dynamic additions.
- **No Batch Processing**: Individual item operations only.
- **No Audit Trail**: Limited tracking of inventory changes and modifications.

2.2 Order Management

- **Basic Order Status**: Simple status tracking without detailed workflow management.
- **No Order Modifications**: Cannot modify or cancel existing orders.
- **Limited Order Types**: Basic order processing without advanced features (partial orders, backordering).
- **No Shipping Integration**: No integration with shipping or logistics systems.

2.3 Reporting Limitations

- **Static Reports**: Basic reporting without customization options.
- **No Data Analytics**: Absence of trend analysis and predictive capabilities.
- **Limited Export Options**: No data export to standard formats (CSV, PDF).
- **No Scheduled Reports**: Manual report generation only.

2.4 User Interface Limitations

- **Console-Based UI**: Text-based interface limiting user experience.
- **No Graphical Elements**: Absence of charts, graphs, and visual indicators.
- **Limited Navigation**: Linear menu navigation without modern UI patterns.
- **No Mobile Support**: Desktop console application only.

3.Business Logic Limitations

3.1 Financial Management

- **No Tax Calculations**: Absence of tax computation and compliance features.
- **Limited Payment Methods**: Basic payment recording without payment gateway integration.
- **No Currency Support**: Single currency operations without multi-currency capabilities.
- **No Financial Analytics**: Basic revenue tracking without detailed financial analysis.

3.2 Supplier Management

- **Basic Supplier Info**: Limited supplier data without comprehensive vendor management.
- **No Purchase Orders**: Absence of formal purchasing workflow.
- **No Supplier Performance**: No tracking of supplier reliability and performance metrics.
- **No Integration**: No integration with supplier systems or APIs.

4. Scalability Limitations

4.1 Data Volume

- **Array Size Limits**: Fixed maximum limits for users, items, and transactions.
- **Memory Constraints**: Static memory allocation limiting growth potential.
- **Performance Degradation**: Linear search operations affecting performance with large datasets.

4.2 Feature Expansion

- **Monolithic Structure**: Single executable limiting modular feature additions.
- **Tight Coupling**: Strong dependencies between components hindering modifications.
- **No Plugin Architecture**: Cannot extend functionality without source code modifications.

REPORT SUMMARY

Overall Project Statistics:

- Total Components: 32
- Complete Implementation: 29 (90.6%)
- Partial Implementation: 3 (9.4%)
- Missing Implementation: 0 (0%)
- Average Performance Rating: Good

Component	Status	Implementation Level	Performance Rating
Admin Login	☑ Complete	Advanced	Excellent
Employee Registration/Login	☑ Complete	Advanced	Excellent
Customer Registration/Login	☑ Complete	Advanced	Excellent
Add/Remove Items	☑ Complete	Advanced	Excellent
Update Items	☑ Complete	Advanced	Good
View Items	☑ Complete	Intermediate	Excellent
Stock Monitoring	☑ Complete	Advanced	Good
Place Orders	☑ Complete	Advanced	Good
Order Status Tracking	☑ Complete	Intermediate	Good
Order History	☑ Complete	Intermediate	Good
Multi-method Payments	☑ Complete	Intermediate	Good
Transaction Logging	☑ Complete	Advanced	Excellent
Receipt Generation	☑ Complete	Advanced	Good
Stock Replenishment	☑ Complete	Intermediate	Good
Supplier Recording	☑ Complete	Basic	Fair
Supply History	☑ Complete	Intermediate	Good
Data Persistence	☑ Complete	Advanced	Good
Backup System	☑ Complete	Basic	Fair
Data Loading	☑ Complete	Advanced	Excellent
Inventory Reports	☑ Complete	Intermediate	Good
Employee Reports	☑ Complete	Intermediate	Good
Low Stock Alerts	☑ Complete	Advanced	Excellent
Menu System	☑ Complete	Advanced	Good

ASCII Art Logo	☑ Complete	Basic	Excellent
Console Formatting	☑ Complete	Advanced	Good
Input Validation	☑ Complete	Intermediate	Good
Business Rule Validation	⚠ Partial	Basic	Fair
Data Integrity	⚠ Partial	Intermediate	Fair
Password Protection	⚠ Partial	Basic	Fair
Session Management	☑ Complete	Basic	Good
Access Control	☑ Complete	Intermediate	Good
Response Time	☑ Complete	Good	Good
Memory Usage	☑ Complete	Good	Good
File I/O Speed	☑ Complete	Good	Fair

Source Code

```
#include <stdio.h>
#include <string.h>
#include <conio.h>
#include <windows.h>
#include <time.h>

#define MAX_USERS 10
#define MAX_ITEMS 50
#define MAX_ORDERS 100
#define MAX_TRANSACTIONS 100
#define LOW_STOCK_THRESHOLD 5

typedef struct {
    char username[30];
    char password[30];
} Admin;

typedef struct {
    char username[30];
```

```
    char password[30];
} Employee;

typedef struct {
    char username[30];
    char password[30];
} Customer;

typedef struct {
    char name[50];
    float price;
    char category[30];
    int quantity;
} Item;

typedef struct {
    int orderId;
    char customerName[30];
    char itemName[50];
    int quantity;
    float totalAmount;
    char status[20];
    char date[20];
} Order;

typedef struct {
    int transactionId;
    char customerName[30];
    char paymentMethod[20];
    char customerId[30];
    char transactionIdStr[50];
    char customerNameFull[50];
    char reference[100];
    float amount;
    char date[20];
} Transaction;

typedef struct {
    char itemName[50];
    int quantitySupplied;
```

```
    char supplierName[50];
    char date[20];
} SupplyRecord;

Admin adminList[] = {
    {"Hamja", "Hamja123"},
    {"Sabbir", "Sabbir123"},
    {"Nafisha", "Nafisha123"}
};
int adminCount = 3;

Employee employeeList[MAX_USERS];
int employeeCount = 0;

Customer customerList[MAX_USERS];
int customerCount = 0;

Item itemList[MAX_ITEMS];
int itemCount = 0;

Order orderList[MAX_ORDERS];
int orderCount = 0;

Transaction transactionList[MAX_TRANSACTIONS];
int transactionCount = 0;

SupplyRecord supplyList[MAX_ITEMS];
int supplyCount = 0;

int adminLoggedIn = 0;
int employeeLoggedIn = 0;
int customerLoggedIn = 0;
char currentCustomer[30] = "";
char currentEmployee[30] = "";

// File handling

void saveDataToFiles();
void loadDataFromFiles();
void saveItems();
```

```
void loadItems();
void saveOrders();
void loadOrders();
void saveTransactions();
void loadTransactions();
void saveSupplyRecords();
void loadSupplyRecords();
void saveEmployees();
void loadEmployees();
void saveCustomers();
void loadCustomers();

int getConsoleWidth() {
    CONSOLE_SCREEN_BUFFER_INFO csbi;
    int columns;
    GetConsoleScreenBufferInfo(GetStdHandle(STD_OUTPUT_HANDLE), &csbi);
    columns = csbi.srWindow.Right - csbi.srWindow.Left + 1;
    return columns;
}

void printCentered(const char *text) {
    int width = getConsoleWidth();
    int len = (int)strlen(text);
    int padding = (width - len) / 2;
    if (padding < 0) padding = 0;
    for (int i = 0; i < padding; i++) printf(" ");
    printf("%s\n", text);
}

void printCenteredInput(const char *text) {
    int width = getConsoleWidth();
    int len = (int)strlen(text);
    int padding = (width - len) / 2;
    if (padding < 0) padding = 0;
    for (int i = 0; i < padding; i++) printf(" ");
    printf("%s", text);
}

void getHiddenPassword(char *password) {
    int i = 0;
```

```
    char ch;
    while (1) {
        ch = getch();
        if (ch == 13) {
            password[i] = '\0';
            printf("\n");
            break;
        } else if (ch == 8) {
            if (i > 0) {
                i--;
                printf("\b \b");
            }
        } else if (i < 29 && ch >= 32 && ch <= 126) {
            password[i++] = ch;
            printf("*");
        }
    }
}

void getCurrentDate(char *dateStr) {
    time_t t = time(NULL);
    struct tm tm = *localtime(&t);
    sprintf(dateStr, "%02d/%02d/%04d", tm.tm_mday, tm.tm_mon + 1, tm.tm_year +
1900);
}

// FILE HANDLING FUNCTIONS

void saveItems() {
    FILE *file = fopen("items.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save items to file!");
        return;
    }


    fprintf(file, "=== ITEMS DATA ===\n");
    fprintf(file, "Total Items: %d\n", itemCount);
    fprintf(file, "Format: Name|Price|Category|Quantity\n");
    fprintf(file, "==================================\n");
```

```
    for (int i = 0; i < itemCount; i++) {
        fprintf(file, "%s|%.2f|%s|%d\n",
                itemList[i].name,
                itemList[i].price,
                itemList[i].category,
                itemList[i].quantity);
    }

    fclose(file);
    printCentered("Items saved to items.txt successfully!");
}

void loadItems() {
    FILE *file = fopen("items.txt", "r");
    if (file == NULL) {
        printCentered("Items file not found - starting with empty inventory.");
        itemCount = 0;
        return;
    }

    char line[200];
    itemCount = 0;


    fgets(line, sizeof(line), file);
    fgets(line, sizeof(line), file);
    fgets(line, sizeof(line), file);
    fgets(line, sizeof(line), file);


    while (fgets(line, sizeof(line), file) && itemCount < MAX_ITEMS) {

        line[strcspn(line, "\n")] = 0;


        char *token = strtok(line, "|");
        if (token != NULL) {
            strcpy(itemList[itemCount].name, token);
```

```
            token = strtok(NULL, "|");
            if (token != NULL) {
                itemList[itemCount].price = atof(token);

                token = strtok(NULL, "|");
                if (token != NULL) {
                    strcpy(itemList[itemCount].category, token);

                    token = strtok(NULL, "|");
                    if (token != NULL) {
                        itemList[itemCount].quantity = atoi(token);
                        itemCount++;
                    }
                }
            }
        }
    }

    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d items from items.txt", itemCount);
    printCentered(loadMsg);
}


void saveOrders() {
    FILE *file = fopen("orders.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save orders to file!");
        return;
    }

    fprintf(file, "=== ORDERS DATA ===\n");
    fprintf(file, "Total Orders: %d\n", orderCount);
    fprintf(file, "Format:
OrderID|CustomerName|ItemName|Quantity|TotalAmount|Status|Date\n");
    fprintf(file,
"==========================================================
===\n");
```

```
    for (int i = 0; i < orderCount; i++) {
        fprintf(file, "%d|%s|%s|%d|%.2f|%s|%s\n",
                orderList[i].orderId,
                orderList[i].customerName,
                orderList[i].itemName,
                orderList[i].quantity,
                orderList[i].totalAmount,
                orderList[i].status,
                orderList[i].date);
    }

    fclose(file);
    printCentered("Orders saved to orders.txt successfully!");
}

void loadOrders() {
    FILE *file = fopen("orders.txt", "r");
    if (file == NULL) {
        printCentered("Orders file not found - starting with empty orders.");
        orderCount = 0;
        return;
    }

    char line[300];
    orderCount = 0;


    for (int i = 0; i < 4; i++) {
        fgets(line, sizeof(line), file);
    }

    while (fgets(line, sizeof(line), file) && orderCount < MAX_ORDERS) {
        line[strcspn(line, "\n")] = 0;

        char *token = strtok(line, "|");
        if (token != NULL) {
            orderList[orderCount].orderId = atoi(token);

            token = strtok(NULL, "|");
```

```
            if (token != NULL) strcpy(orderList[orderCount].customerName, token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(orderList[orderCount].itemName, token);

            token = strtok(NULL, "|");
            if (token != NULL) orderList[orderCount].quantity = atoi(token);

            token = strtok(NULL, "|");
            if (token != NULL) orderList[orderCount].totalAmount = atof(token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(orderList[orderCount].status, token);

            token = strtok(NULL, "|");
            if (token != NULL) {
                strcpy(orderList[orderCount].date, token);
                orderCount++;
            }
        }
    }

    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d orders from orders.txt", orderCount);
    printCentered(loadMsg);
}


void saveTransactions() {
    FILE *file = fopen("transactions.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save transactions to file!");
        return;
    }

    fprintf(file, "=== TRANSACTIONS DATA ===\n");
    fprintf(file, "Total Transactions: %d\n", transactionCount);
    fprintf(file, "Format:
TransactionID|CustomerName|PaymentMethod|CustomerID|TransactionIDStr|Custom
```

```
erNameFull|Reference|Amount|Date\n");
    fprintf(file,
"=============================================================
=========================================\n");

    for (int i = 0; i < transactionCount; i++) {
        fprintf(file, "%d|%s|%s|%s|%s|%s|%s|%.2f|%s\n",
                transactionList[i].transactionId,
                transactionList[i].customerName,
                transactionList[i].paymentMethod,
                transactionList[i].customerId,
                transactionList[i].transactionIdStr,
                transactionList[i].customerNameFull,
                transactionList[i].reference,
                transactionList[i].amount,
                transactionList[i].date);
    }

    fclose(file);
    printCentered("Transactions saved to transactions.txt successfully!");
}

void loadTransactions() {
    FILE *file = fopen("transactions.txt", "r");
    if (file == NULL) {
        printCentered("Transactions file not found - starting with empty transactions.");
        transactionCount = 0;
        return;
    }

    char line[500];
    transactionCount = 0;


    for (int i = 0; i < 4; i++) {
        fgets(line, sizeof(line), file);
    }

    while (fgets(line, sizeof(line), file) && transactionCount <
MAX_TRANSACTIONS) {
```

```
        line[strcspn(line, "\n")] = 0;

        char *token = strtok(line, "|");
        if (token != NULL) {
            transactionList[transactionCount].transactionId = atoi(token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(transactionList[transactionCount].customerName,
token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(transactionList[transactionCount].paymentMethod,
token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(transactionList[transactionCount].customerId,
token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(transactionList[transactionCount].transactionIdStr,
token);

            token = strtok(NULL, "|");
            if (token != NULL)
strcpy(transactionList[transactionCount].customerNameFull, token);

            token = strtok(NULL, "|");
            if (token != NULL) strcpy(transactionList[transactionCount].reference, token);

            token = strtok(NULL, "|");
            if (token != NULL) transactionList[transactionCount].amount = atof(token);

            token = strtok(NULL, "|");
            if (token != NULL) {
                strcpy(transactionList[transactionCount].date, token);
                transactionCount++;
            }
        }
    }
```

```
    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d transactions from transactions.txt", transactionCount);
    printCentered(loadMsg);
}


void saveSupplyRecords() {
    FILE *file = fopen("supply_records.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save supply records to file!");
        return;
    }

    fprintf(file, "=== SUPPLY RECORDS DATA ===\n");
    fprintf(file, "Total Supply Records: %d\n", supplyCount);
    fprintf(file, "Format: ItemName|QuantitySupplied|SupplierName|Date\n");
    fprintf(file, "==========================================\n");

    for (int i = 0; i < supplyCount; i++) {
        fprintf(file, "%s|%d|%s|%s\n",
                supplyList[i].itemName,
                supplyList[i].quantitySupplied,
                supplyList[i].supplierName,
                supplyList[i].date);
    }

    fclose(file);
    printCentered("Supply records saved to supply_records.txt successfully!");
}

void loadSupplyRecords() {
    FILE *file = fopen("supply_records.txt", "r");
    if (file == NULL) {
        printCentered("Supply records file not found - starting with empty records.");
        supplyCount = 0;
        return;
    }

    char line[200];
```

```
    supplyCount = 0;


    for (int i = 0; i < 4; i++) {
        fgets(line, sizeof(line), file);
    }

    while (fgets(line, sizeof(line), file) && supplyCount < MAX_ITEMS) {
        line[strcspn(line, "\n")] = 0;

        char *token = strtok(line, "|");
        if (token != NULL) {
            strcpy(supplyList[supplyCount].itemName, token);

            token = strtok(NULL, "|");
            if (token != NULL) {
                supplyList[supplyCount].quantitySupplied = atoi(token);

                token = strtok(NULL, "|");
                if (token != NULL) {
                    strcpy(supplyList[supplyCount].supplierName, token);

                    token = strtok(NULL, "|");
                    if (token != NULL) {
                        strcpy(supplyList[supplyCount].date, token);
                        supplyCount++;
                    }
                }
            }
        }
    }

    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d supply records from supply_records.txt",
supplyCount);
    printCentered(loadMsg);
}
```

```
void saveEmployees() {
    FILE *file = fopen("employees.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save employees to file!");
        return;
    }

    fprintf(file, "=== EMPLOYEES DATA ===\n");
    fprintf(file, "Total Employees: %d\n", employeeCount);
    fprintf(file, "Format: Username|Password\n");
    fprintf(file, "========================\n");

    for (int i = 0; i < employeeCount; i++) {
        fprintf(file, "%s|%s\n",
                employeeList[i].username,
                employeeList[i].password);
    }

    fclose(file);
    printCentered("Employees saved to employees.txt successfully!");
}

void loadEmployees() {
    FILE *file = fopen("employees.txt", "r");
    if (file == NULL) {
        printCentered("Employees file not found - starting with empty employee list.");
        employeeCount = 0;
        return;
    }

    char line[100];
    employeeCount = 0;


    for (int i = 0; i < 4; i++) {
        fgets(line, sizeof(line), file);
    }

    while (fgets(line, sizeof(line), file) && employeeCount < MAX_USERS) {
        line[strcspn(line, "\n")] = 0;
```

```
        char *token = strtok(line, "|");
        if (token != NULL) {
            strcpy(employeeList[employeeCount].username, token);

            token = strtok(NULL, "|");
            if (token != NULL) {
                strcpy(employeeList[employeeCount].password, token);
                employeeCount++;
            }
        }
    }

    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d employees from employees.txt", employeeCount);
    printCentered(loadMsg);
}


void saveCustomers() {
    FILE *file = fopen("customers.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save customers to file!");
        return;
    }

    fprintf(file, "=== CUSTOMERS DATA ===\n");
    fprintf(file, "Total Customers: %d\n", customerCount);
    fprintf(file, "Format: Username|Password\n");
    fprintf(file, "========================\n");

    for (int i = 0; i < customerCount; i++) {
        fprintf(file, "%s|%s\n",
                customerList[i].username,
                customerList[i].password);
    }

    fclose(file);
    printCentered("Customers saved to customers.txt successfully!");
```

```
}

void loadCustomers() {
    FILE *file = fopen("customers.txt", "r");
    if (file == NULL) {
        printCentered("Customers file not found - starting with empty customer list.");
        customerCount = 0;
        return;
    }

    char line[100];
    customerCount = 0;


    for (int i = 0; i < 4; i++) {
        fgets(line, sizeof(line), file);
    }

    while (fgets(line, sizeof(line), file) && customerCount < MAX_USERS) {
        line[strcspn(line, "\n")] = 0;

        char *token = strtok(line, "|");
        if (token != NULL) {
            strcpy(customerList[customerCount].username, token);

            token = strtok(NULL, "|");
            if (token != NULL) {
                strcpy(customerList[customerCount].password, token);
                customerCount++;
            }
        }
    }

    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d customers from customers.txt", customerCount);
    printCentered(loadMsg);
}
```

```
void saveAdmins() {
    FILE *file = fopen("admins.txt", "w");
    if (file == NULL) {
        printCentered("Error: Could not save admins to file!");
        return;
    }

    fprintf(file, "=== ADMINS DATA ===\n");
    fprintf(file, "Total Admins: %d\n", adminCount);
    fprintf(file, "Format: Username|Password\n");
    fprintf(file, "====================\n");

    for (int i = 0; i < adminCount; i++) {
        fprintf(file, "%s|%s\n",
                adminList[i].username,
                adminList[i].password);
    }

    fclose(file);
    printCentered("Admins saved to admins.txt successfully!");
}

void loadAdmins() {
    FILE *file = fopen("admins.txt", "r");
    if (file == NULL) {
        printCentered("Admins file not found - using default admin accounts.");
        return;
    }

    char line[100];
    adminCount = 0;


    for (int i = 0; i < 4; i++) {
        fgets(line, sizeof(line), file);
    }

    while (fgets(line, sizeof(line), file) && adminCount < MAX_USERS) {
        line[strcspn(line, "\n")] = 0;
```

```
        char *token = strtok(line, "|");
        if (token != NULL) {
            strcpy(adminList[adminCount].username, token);

            token = strtok(NULL, "|");
            if (token != NULL) {
                strcpy(adminList[adminCount].password, token);
                adminCount++;
            }
        }
    }

    fclose(file);
    char loadMsg[100];
    sprintf(loadMsg, "Loaded %d admins from admins.txt", adminCount);
    printCentered(loadMsg);
}


void saveDataToFiles() {
    printCentered("Saving all data to text files...");
    saveItems();
    saveOrders();
    saveTransactions();
    saveSupplyRecords();
    saveEmployees();
    saveCustomers();
    saveAdmins();
    printCentered("All data saved successfully to .txt files!");
}

void loadDataFromFiles() {
    printCentered("Loading data from text files...");
    loadItems();
    loadOrders();
    loadTransactions();
    loadSupplyRecords();
    loadEmployees();
    loadCustomers();
    loadAdmins();
```

```
    printCentered("All data loaded successfully from .txt files!");
}


void createBackupFiles() {
    char timestamp[50];
    getCurrentDate(timestamp);


    for (int i = 0; timestamp[i]; i++) {
        if (timestamp[i] == '/') timestamp[i] = '_';
    }

    char backupFolder[100];
    sprintf(backupFolder, "backup_%s", timestamp);


    #ifdef _WIN32
    char command[150];
    sprintf(command, "mkdir \"%s\" 2>nul", backupFolder);
    system(command);


    sprintf(command, "copy *.txt \"%s\\\" 2>nul", backupFolder);
    system(command);
    #endif

    char backupMsg[200];
    sprintf(backupMsg, "Backup created in folder: %s", backupFolder);
    printCentered(backupMsg);
}


int validateAllData() {
    int errors = 0;


    for (int i = 0; i < itemCount; i++) {
        if (itemList[i].price < 0 || itemList[i].quantity < 0 ||
            strlen(itemList[i].name) == 0) {
```

```
            errors++;
        }
    }


    for (int i = 0; i < orderCount; i++) {
        if (orderList[i].quantity <= 0 || orderList[i].totalAmount < 0 ||
            strlen(orderList[i].customerName) == 0 || strlen(orderList[i].itemName) == 0) {
            errors++;
        }
    }


    for (int i = 0; i < transactionCount; i++) {
        if (transactionList[i].amount <= 0 ||
            strlen(transactionList[i].customerName) == 0) {
            errors++;
        }
    }

    if (errors > 0) {
        char errorMsg[100];
        sprintf(errorMsg, "Data validation failed! Found %d errors.", errors);
        printCentered(errorMsg);
        return 0;
    } else {
        printCentered("Data validation successful!");
        return 1;
    }
}

void displayFullAsciiLogo() {
    const char *logo[] = {
        ".....",
        ".....",
        ".....:-=+*#####+=-:.....",
        ".....=#%%%%%%%%%%%%%%%%%%#=.....::.....",
        ".....=%%%%%%*:.....:+%%%%%%%+.*%%.....",
        ".....:*%%%%%+:.....=%%%%%%%%*.....",
        ".....*%%%#:.....::::.....#%%%%%%.....",
```

"....................:%%%#.......:#####=...........:%%%%%%%%*....................",
"....................=%%%-........:#####=.................:-=++.....................",
"...................=%%%:.........:#####=.........:#%%-.........................",
"..................-%%%:...+****+.:#####=......-%%%%##%%%+:.....................",
"..................%%%:....*#####.:#####=...+%%%#*******#%%%#-....................",

".................=%%#.....*#####.:###*::#%%%#***************%%%%=.................",
"................#%%:.....*#####.:#:-
#%%%#************=..*****#%%%+:.............",

".................%%%......*#####.:*.%%%#************+..:*******#%%%*.............",

".................%%%####=.*#####.:*.%%#%%%#***=..+*:..******#%%%%#%*.....
...",
"................%%%####=.*#####.:*.%%***#%%%#**:....-
****#%%%#***#%*.............",
"................#%%####=.*#####.:*.%%******#%%%%*-
#%%%#***#%*.............",

".................=%%%###=.*#####.:*.%%**********%%%%%%%#*********#%*.....
.......",

".................%%%###=.*#####.:*.%%***********#%%***********#%*.............
",
"................-
%%%##=.*#####.:*.%%***********#%%***********#%*.............",
"................=%%%*-
.=++++.:+.%%%**********#%%**********#%%+.............",
"..................+%%%-
.............:*%%%#*******#%%********#%%%=.............",
"...................-%%%*.............:*%%%*****#%%*****#%%%-.................",
"....................#%%%=.............:*%%%#*#%%**#%%#-.....................",
"....................:%%%%#:.............:+%%%%%%%#-.......................",
"......................%%%%%%=:...........:-.=%#:.........................",
"......................-
*%%%%%%%%%%%%%%%%%%%:.........................",
"..........................:=#%%%%%%%%%*=:...............................",
"...",
"...",
".....+%%%%:%%%%%@:+%%%%#..-

```
%%%%%:%%.+%+.....=%%:..+%*..*%:+%%%%#:.....",
        ".....*%=:....%%..*%:...##:%#.....%%%%-
.....:%+*%..+%#%.*%:+%...#%.....",
        ".....:=#%-..%%..*%:...#%:%*.....%%-
%#.....#%+*%+.+%.*%#%:+%...#%.....",
        ".....:%%%%%-..%%...#%%%%%:.-%%%%%:%%..%%...=%-..=%:+%..-
%%:+%%%%%:.....",
        "................................................................................",
        ".....:+=.+=..:+-=+:..:+--++++=.+-..:+:=+++++:.:+*+:..=+++-.=*:..+*:.....",
        ".....:%*:%%#.-%+.%*..%%:+%:....%%*.:%-
..=%..:%%:.=%*.*%..%%.*%.*%-.....",
        ".....:%*:%**%-%+.=%--%:.+%%%%:.%+#%:%-..=%..+%-...+%-
*%*#%*..#%%-.....",
        ".....:%*:%*.+%%+..##%#..+%:....%+.+%%-
..=%..:%*...%#.*%.#%:..:%+.....",
        ".....:%+:%+..-%+..:%%...+%%%%*.%=..-%-..=%....+%%%-
..*#..#%:.:%+.....",
        "................................................................................",
        "................................................................................",
    };


    int lines = sizeof(logo) / sizeof(logo[0]);

    printf("\033[1;36m");
    for (int i = 0; i < lines; i++) {
        printCentered(logo[i]);
    }
    printf("\033[0m");
}

const char *categories[] = {
    "Phone", "Laptop", "Monitor", "Electronics",
    "Fruit", "Vegetable", "Meat", "Drinks"
};
#define CATEGORY_COUNT (sizeof(categories) / sizeof(categories[0]))

// Functions of Admin

void viewAllItems() {
```

```
    if (itemCount == 0) {
        printCentered("");
        printCentered("No items to display.");
        return;
    }
    printCentered("");
    printCentered("---------- List of Items ----------");
    for (int i = 0; i < itemCount; i++) {
        char itemInfo[200];
        if (itemList[i].quantity <= LOW_STOCK_THRESHOLD) {
            sprintf(itemInfo, "%d. %s | Category: %s | $%.2f | Stock: %d [LOW
STOCK!]",
                    i + 1,
                    itemList[i].name,
                    itemList[i].category,
                    itemList[i].price,
                    itemList[i].quantity);
        } else {
            sprintf(itemInfo, "%d. %s | Category: %s | $%.2f | Stock: %d",
                    i + 1,
                    itemList[i].name,
                    itemList[i].category,
                    itemList[i].price,
                    itemList[i].quantity);
        }
        printCentered(itemInfo);
    }
}

void addItem() {
    if (itemCount >= MAX_ITEMS) {
        printCentered("Item storage is full!");
        return;
    }

    printCentered("");
    printCentered("Add New Item");
    printCenteredInput("Enter item name: ");
    scanf(" %[^\n]", itemList[itemCount].name);
```

```
    printCenteredInput("Enter item price: ");
    scanf("%f", &itemList[itemCount].price);

    printCentered("");
    printCentered("Select a category:");
    for (int i = 0; i < CATEGORY_COUNT; i++) {
        char categoryOption[50];
        sprintf(categoryOption, "%d. %s", i + 1, categories[i]);
        printCentered(categoryOption);
    }

    int categoryChoice;
    printCenteredInput("Enter category number: ");
    scanf("%d", &categoryChoice);

    if (categoryChoice < 1 || categoryChoice > (int)CATEGORY_COUNT) {
        printCentered("Invalid category! Setting to 'Uncategorized'");
        strcpy(itemList[itemCount].category, "Uncategorized");
    } else {
        strcpy(itemList[itemCount].category, categories[categoryChoice - 1]);
    }

    printCenteredInput("Enter quantity: ");
    scanf("%d", &itemList[itemCount].quantity);
    if (itemList[itemCount].quantity < 0) itemList[itemCount].quantity = 0;

    itemCount++;
    saveItems();
    printCentered("Item added successfully and saved to file!");
}

void removeItem() {
    if (itemCount == 0) {
        printCentered("No items to remove.");
        return;
    }

    printCentered("");
    printCentered("---------- Remove Item ----------");
    viewAllItems();
```

```
    int index;
    printCenteredInput("Enter the item number to remove: ");
    scanf("%d", &index);

    if (index < 1 || index > itemCount) {
        printCentered("Invalid item number.");
        return;
    }

    index--;

    for (int i = index; i < itemCount - 1; i++) {
        itemList[i] = itemList[i + 1];
    }

    itemCount--;
    saveItems();
    printCentered("Item removed successfully and changes saved to file!");
}

void updateItem() {
    if (itemCount == 0) {
        printCentered("No items to update.");
        return;
    }

    printCentered("");
    printCentered("---------- Update Item ----------");
    viewAllItems();

    int index;
    printCenteredInput("Enter item number to update: ");
    scanf("%d", &index);

    if (index < 1 || index > itemCount) {
        printCentered("Invalid item number.");
        return;
    }
```

```
index--;

char updateInfo[200];
sprintf(updateInfo, "Updating item: %s ($%.2f, %s, stock %d)",
      itemList[index].name,
      itemList[index].price,
      itemList[index].category,
      itemList[index].quantity);
printCentered(updateInfo);

printCenteredInput("Enter new name: ");
char newName[50];
scanf(" %[^\n]", newName);
if (strcmp(newName, ".") != 0) {
   strcpy(itemList[index].name, newName);
}

printCenteredInput("Enter new price: ");
float newPrice;
scanf("%f", &newPrice);
if (newPrice >= 0) {
   itemList[index].price = newPrice;
}

printCentered("");
printCentered("Select new category:");
for (int i = 0; i < CATEGORY_COUNT; i++) {
   char categoryOption[50];
   sprintf(categoryOption, "%d. %s", i + 1, categories[i]);
   printCentered(categoryOption);
}
int categoryChoice;
printCenteredInput("Enter category number: ");
scanf("%d", &categoryChoice);
if (categoryChoice >= 1 && categoryChoice <= (int)CATEGORY_COUNT) {
   strcpy(itemList[index].category, categories[categoryChoice - 1]);
}

printCenteredInput("Enter new quantity : ");
int newQty;
```

```
    scanf("%d", &newQty);
    if (newQty >= 0) {
        itemList[index].quantity = newQty;
    }

    saveItems();
    printCentered("Item updated successfully and changes saved to file!");
}

void viewItemsByCategory() {
    if (itemCount == 0) {
        printCentered("");
        printCentered("No items in the inventory.");
        return;
    }

    printCentered("");
    printCentered("Select category to view:");
    for (int i = 0; i < CATEGORY_COUNT; i++) {
        char categoryOption[50];
        sprintf(categoryOption, "%d. %s", i + 1, categories[i]);
        printCentered(categoryOption);
    }

    int choice;
    printCenteredInput("Enter category number: ");
    scanf("%d", &choice);

    if (choice < 1 || choice > (int)CATEGORY_COUNT) {
        printCentered("Invalid category choice!");
        return;
    }

    char categoryHeader[100];
    sprintf(categoryHeader, "---------- Items in Category: %s ----------",
categories[choice - 1]);
    printCentered("");
    printCentered(categoryHeader);

    int found = 0;
```

```
    for (int i = 0; i < itemCount; i++) {
        if (strcmp(itemList[i].category, categories[choice - 1]) == 0) {
            char itemInfo[200];
            if (itemList[i].quantity <= LOW_STOCK_THRESHOLD) {
                sprintf(itemInfo, "%d. %s | $%.2f | Stock: %d [LOW STOCK!]",
                        i + 1, itemList[i].name, itemList[i].price, itemList[i].quantity);
            } else {
                sprintf(itemInfo, "%d. %s | $%.2f | Stock: %d",
                        i + 1, itemList[i].name, itemList[i].price, itemList[i].quantity);
            }
            printCentered(itemInfo);
            found = 1;
        }
    }

    if (!found) {
        printCentered("No items found in this category.");
    }
}

// Functions Of Employee

void supplyProduct() {
    if (itemCount == 0) {
        printCentered("No items in inventory to supply.");
        return;
    }

    printCentered("");
    printCentered("---------- Supply Product ----------");
    viewAllItems();

    int itemIndex;
    printCenteredInput("Enter item number to supply: ");
    scanf("%d", &itemIndex);

    if (itemIndex < 1 || itemIndex > itemCount) {
        printCentered("Invalid item number!");
        return;
    }
```

```
    itemIndex--;

    int quantity;
    printCenteredInput("Enter quantity to supply: ");
    scanf("%d", &quantity);

    if (quantity <= 0) {
        printCentered("Invalid quantity!");
        return;
    }

    char supplier[50];
    printCenteredInput("Enter supplier name: ");
    scanf(" %[^\n]", supplier);

    itemList[itemIndex].quantity += quantity;

    strcpy(supplyList[supplyCount].itemName, itemList[itemIndex].name);
    supplyList[supplyCount].quantitySupplied = quantity;
    strcpy(supplyList[supplyCount].supplierName, supplier);
    getCurrentDate(supplyList[supplyCount].date);
    supplyCount++;


    saveItems();
    saveSupplyRecords();

    char successMsg[100];
    sprintf(successMsg, "Product supplied successfully! New stock: %d",
itemList[itemIndex].quantity);
    printCentered(successMsg);
    printCentered("Supply record saved to file!");
}

void receiveStock() {
    printCentered("");
    printCentered("---------- Stock Alert Report ----------");
    char alertMsg[100];
    sprintf(alertMsg, "Items with low stock :", LOW_STOCK_THRESHOLD);
```

```
    printCentered(alertMsg);
    printCentered("");

    int lowStockCount = 0;
    for (int i = 0; i < itemCount; i++) {
        if (itemList[i].quantity <= LOW_STOCK_THRESHOLD) {
            char alertInfo[200];
            sprintf(alertInfo, "ALERT: %s | Current Stock: %d | Category: %s",
                    itemList[i].name, itemList[i].quantity, itemList[i].category);
            printCentered(alertInfo);
            lowStockCount++;
        }
    }

    if (lowStockCount == 0) {
        printCentered("All items have sufficient stock!");
    } else {
        char totalMsg[100];
        sprintf(totalMsg, "Total items needing restock: %d", lowStockCount);
        printCentered("");
        printCentered(totalMsg);
    }
}

void receivePayment() {
    if (transactionCount == 0) {
        printCentered("No transactions to process.");
        return;
    }

    printCentered("");
    printCentered("---------- Pending Payments ----------");
    for (int i = 0; i < transactionCount; i++) {
        char paymentInfo[200];
        sprintf(paymentInfo, "%d. Customer: %s | Amount: $%.2f | Method: %s | Date:
%s",
                i + 1, transactionList[i].customerName, transactionList[i].amount,
                transactionList[i].paymentMethod, transactionList[i].date);
        printCentered(paymentInfo);
    }
```

```
    printCentered("");
    printCentered("All payments have been processed and recorded.");
}

void generateEmployeeReport() {
    char date[20];
    getCurrentDate(date);

    printCentered("");
    printCentered("================== EMPLOYEE REPORT
==================");
    char reportHeader1[100], reportHeader2[100];
    sprintf(reportHeader1, "Generated by: %s", currentEmployee);
    sprintf(reportHeader2, "Date: %s", date);
    printCentered(reportHeader1);
    printCentered(reportHeader2);
    printCentered("");

    printCentered("---------- INVENTORY SUMMARY ----------");
    char totalItemsMsg[50];
    sprintf(totalItemsMsg, "Total Items: %d", itemCount);
    printCentered(totalItemsMsg);

    float totalValue = 0;
    for (int i = 0; i < itemCount; i++) {
        totalValue += itemList[i].price * itemList[i].quantity;
    }
    char totalValueMsg[100];
    sprintf(totalValueMsg, "Total Inventory Value: $%.2f", totalValue);
    printCentered(totalValueMsg);
    printCentered("");

    printCentered("---------- LOW STOCK ALERTS ----------");
    int lowStockItems = 0;
    for (int i = 0; i < itemCount; i++) {
        if (itemList[i].quantity <= LOW_STOCK_THRESHOLD) {
            char lowStockMsg[100];
            sprintf(lowStockMsg, "• %s: %d units remaining", itemList[i].name,
itemList[i].quantity);
            printCentered(lowStockMsg);
```

```
            lowStockItems++;
        }
    }
    if (lowStockItems == 0) {
        printCentered("No items with low stock.");
    }
    printCentered("");

    printCentered("---------- RECENT SUPPLY RECORDS ----------");
    if (supplyCount == 0) {
        printCentered("No supply records available.");
    } else {
        for (int i = 0; i < supplyCount; i++) {
            char supplyMsg[200];
            sprintf(supplyMsg, "• %s: %d units from %s on %s",
                    supplyList[i].itemName, supplyList[i].quantitySupplied,
                    supplyList[i].supplierName, supplyList[i].date);
            printCentered(supplyMsg);
        }
    }
    printCentered("");

    printCentered("---------- TRANSACTION SUMMARY ----------");
    char transactionCountMsg[50];
    sprintf(transactionCountMsg, "Total Transactions: %d", transactionCount);
    printCentered(transactionCountMsg);

    float totalRevenue = 0;
    for (int i = 0; i < transactionCount; i++) {
        totalRevenue += transactionList[i].amount;
    }
    char revenueMsg[100];
    sprintf(revenueMsg, "Total Revenue: $%.2f", totalRevenue);
    printCentered(revenueMsg);


printCentered("================================================");


    FILE *reportFile = fopen("employee_report.txt", "w");
```

```
    if (reportFile != NULL) {
        fprintf(reportFile, "================== EMPLOYEE REPORT
==================\n");
        fprintf(reportFile, "Generated by: %s\n", currentEmployee);
        fprintf(reportFile, "Date: %s\n\n", date);

        fprintf(reportFile, "---------- INVENTORY SUMMARY ----------\n");
        fprintf(reportFile, "Total Items: %d\n", itemCount);
        fprintf(reportFile, "Total Inventory Value: $%.2f\n\n", totalValue);

        fprintf(reportFile, "---------- LOW STOCK ALERTS ----------\n");
        if (lowStockItems == 0) {
            fprintf(reportFile, "No items with low stock.\n");
        } else {
            for (int i = 0; i < itemCount; i++) {
                if (itemList[i].quantity <= LOW_STOCK_THRESHOLD) {
                    fprintf(reportFile, "• %s: %d units remaining\n", itemList[i].name,
itemList[i].quantity);
                }
            }
        }

        fprintf(reportFile, "\n---------- TRANSACTION SUMMARY ----------\n");
        fprintf(reportFile, "Total Transactions: %d\n", transactionCount);
        fprintf(reportFile, "Total Revenue: $%.2f\n", totalRevenue);

        fclose(reportFile);
        printCentered("Report saved to 'employee_report.txt'!");
    }
}

// Functions of Customer

void placeOrder() {
    if (!customerLoggedIn) {
        printCentered("Please login as customer to place order.");
        return;
    }

    if (itemCount == 0) {
```

```
    printCentered("No items available for order.");
    return;
}

printCentered("");
printCentered("---------- Place Order ----------");
viewAllItems();

int itemChoice, quantity;
printCenteredInput("Enter item number to order: ");
scanf("%d", &itemChoice);

if (itemChoice < 1 || itemChoice > itemCount) {
    printCentered("Invalid item number!");
    return;
}

itemChoice--;
Item *selectedItem = &itemList[itemChoice];

char selectedInfo[200];
sprintf(selectedInfo, "Selected: %s | Price: $%.2f | Available: %d",
        selectedItem->name, selectedItem->price, selectedItem->quantity);
printCentered(selectedInfo);

printCenteredInput("Enter quantity to order: ");
scanf("%d", &quantity);

if (quantity <= 0 || quantity > selectedItem->quantity) {
    char errorMsg[100];
    sprintf(errorMsg, "Invalid quantity! Available: %d", selectedItem->quantity);
    printCentered(errorMsg);
    return;
}

orderList[orderCount].orderId = orderCount + 1;
strcpy(orderList[orderCount].customerName, currentCustomer);
strcpy(orderList[orderCount].itemName, selectedItem->name);
orderList[orderCount].quantity = quantity;
orderList[orderCount].totalAmount = quantity * selectedItem->price;
```

```
    strcpy(orderList[orderCount].status, "Pending");
    getCurrentDate(orderList[orderCount].date);

    printCentered("");
    printCentered("---------- Order Details ----------");
    char orderDetails[200];
    sprintf(orderDetails, "Order ID: %d", orderList[orderCount].orderId);
    printCentered(orderDetails);
    sprintf(orderDetails, "Customer: %s", orderList[orderCount].customerName);
    printCentered(orderDetails);
    sprintf(orderDetails, "Item: %s", orderList[orderCount].itemName);
    printCentered(orderDetails);
    sprintf(orderDetails, "Quantity: %d", orderList[orderCount].quantity);
    printCentered(orderDetails);
    sprintf(orderDetails, "Total Amount: $%.2f", orderList[orderCount].totalAmount);
    printCentered(orderDetails);
    sprintf(orderDetails, "Status: %s", orderList[orderCount].status);
    printCentered(orderDetails);
    sprintf(orderDetails, "Date: %s", orderList[orderCount].date);
    printCentered(orderDetails);

    orderCount++;
    saveOrders();
    printCentered("");
    printCentered("Order placed successfully and saved to file!");


    char filename[100];
    sprintf(filename, "order_%s_%d.txt", currentCustomer, orderList[orderCount-
1].orderId);
    FILE *receiptFile = fopen(filename, "w");

    if (receiptFile != NULL) {
        fprintf(receiptFile, "========== ORDER RECEIPT ==========\n");
        fprintf(receiptFile, "Order ID: %d\n", orderList[orderCount-1].orderId);
        fprintf(receiptFile, "Customer: %s\n", orderList[orderCount-1].customerName);
        fprintf(receiptFile, "Item: %s\n", orderList[orderCount-1].itemName);
        fprintf(receiptFile, "Quantity: %d\n", orderList[orderCount-1].quantity);
        fprintf(receiptFile, "Unit Price: $%.2f\n", selectedItem->price);
        fprintf(receiptFile, "Total Amount: $%.2f\n", orderList[orderCount-
```

```
1].totalAmount);
        fprintf(receiptFile, "Status: %s\n", orderList[orderCount-1].status);
        fprintf(receiptFile, "Date: %s\n", orderList[orderCount-1].date);
        fprintf(receiptFile, "==================================\n");
        fclose(receiptFile);

        char receiptMsg[150];
        sprintf(receiptMsg, "Order receipt saved as '%s'", filename);
        printCentered(receiptMsg);
    }
}

void receiveOrder() {
    if (!customerLoggedIn) {
        printCentered("Please login as customer.");
        return;
    }

    printCentered("");
    printCentered("---------- Your Orders ----------");
    int found = 0;

    for (int i = 0; i < orderCount; i++) {
        if (strcmp(orderList[i].customerName, currentCustomer) == 0) {
            char orderInfo[200];
            sprintf(orderInfo, "Order ID: %d | Item: %s | Qty: %d | Amount: $%.2f | Status:
%s | Date: %s",
                    orderList[i].orderId, orderList[i].itemName, orderList[i].quantity,
                    orderList[i].totalAmount, orderList[i].status, orderList[i].date);
            printCentered(orderInfo);
            found = 1;
        }
    }

    if (!found) {
        char noOrderMsg[100];
        sprintf(noOrderMsg, "No orders found for %s.", currentCustomer);
        printCentered(noOrderMsg);
        return;
    }
```

```
    int orderChoice;
    printCenteredInput("Enter Order ID to receive : ");
    scanf("%d", &orderChoice);


    if (orderChoice == 0) return;

    for (int i = 0; i < orderCount; i++) {
        if (orderList[i].orderId == orderChoice &&
            strcmp(orderList[i].customerName, currentCustomer) == 0) {

            if (strcmp(orderList[i].status, "Completed") == 0) {
                printCentered("Order already completed!");
                return;
            }

            strcpy(orderList[i].status, "Received");
            saveOrders();

            char successMsg[100];
            sprintf(successMsg, "Order %d marked as received!", orderChoice);
            printCentered(successMsg);
            printCentered("Order status updated and saved to file!");
            return;
        }
    }
    printCentered("Order not found!");
}

void processPayment() {
    if (!customerLoggedIn) {
        printCentered("Please login as customer.");
        return;
    }


    printCentered("");
    printCentered("---------- Pending Orders for Payment ----------");
    int found = 0;
```

```
    float totalAmount = 0;

    for (int i = 0; i < orderCount; i++) {
        if (strcmp(orderList[i].customerName, currentCustomer) == 0 &&
            (strcmp(orderList[i].status, "Pending") == 0 || strcmp(orderList[i].status,
"Received") == 0)) {
            char orderInfo[200];
            sprintf(orderInfo, "Order ID: %d | Item: %s | Amount: $%.2f",
                orderList[i].orderId, orderList[i].itemName, orderList[i].totalAmount);
            printCentered(orderInfo);
            totalAmount += orderList[i].totalAmount;
            found = 1;
        }
    }

    if (!found) {
        printCentered("No pending orders for payment.");
        return;
    }


    char totalMsg[100];
    sprintf(totalMsg, "Total Amount to Pay: $%.2f", totalAmount);
    printCentered("");
    printCentered(totalMsg);

    printCentered("");
    printCentered("---------- Select Payment Method ----------");
    printCentered("1. Visa");
    printCentered("2. PayPal");
    printCentered("3. Bkash");
    printCentered("4. Nagad");
    printCenteredInput("Choose payment method: ");

    int methodChoice;
    scanf("%d", &methodChoice);


    char paymentMethod[20];
    switch (methodChoice) {
```

```
        case 1: strcpy(paymentMethod, "Visa"); break;
        case 2: strcpy(paymentMethod, "PayPal"); break;
        case 3: strcpy(paymentMethod, "Bkash"); break;
        case 4: strcpy(paymentMethod, "Nagad"); break;
        default:
            printCentered("Invalid payment method!");
            return;
    }

    transactionList[transactionCount].transactionId = transactionCount + 1;
    strcpy(transactionList[transactionCount].customerName, currentCustomer);
    strcpy(transactionList[transactionCount].paymentMethod, paymentMethod);

    printCenteredInput("Enter Customer ID/Number: ");
    scanf(" %[^\n]", transactionList[transactionCount].customerId);

    printCenteredInput("Enter Transaction ID: ");
    scanf(" %[^\n]", transactionList[transactionCount].transactionIdStr);

    printCenteredInput("Enter Full Name: ");
    scanf(" %[^\n]", transactionList[transactionCount].customerNameFull);

    printCenteredInput("Enter Reference (optional): ");
    scanf(" %[^\n]", transactionList[transactionCount].reference);

    transactionList[transactionCount].amount = totalAmount;
    getCurrentDate(transactionList[transactionCount].date);


    for (int i = 0; i < orderCount; i++) {
        if (strcmp(orderList[i].customerName, currentCustomer) == 0 &&
            (strcmp(orderList[i].status, "Pending") == 0 || strcmp(orderList[i].status,
"Received") == 0)) {

            strcpy(orderList[i].status, "Completed");

            for (int j = 0; j < itemCount; j++) {
                if (strcmp(itemList[j].name, orderList[i].itemName) == 0) {
                    itemList[j].quantity -= orderList[i].quantity;
                    break;
```

```
            }
        }
    }
}

transactionCount++;


saveTransactions();
saveOrders();
saveItems();


printCentered("");
printCentered("---------- Payment Confirmation ----------");
char confirmationInfo[200];
sprintf(confirmationInfo, "Transaction ID: %d", transactionList[transactionCount-
1].transactionId);
printCentered(confirmationInfo);
sprintf(confirmationInfo, "Payment Method: %s", paymentMethod);
printCentered(confirmationInfo);
sprintf(confirmationInfo, "Amount Paid: $%.2f", totalAmount);
printCentered(confirmationInfo);
sprintf(confirmationInfo, "Date: %s", transactionList[transactionCount-1].date);
printCentered(confirmationInfo);
printCentered("Payment successful! All orders completed.");
printCentered("Transaction data saved to file!");


char receiptFilename[100];
sprintf(receiptFilename, "payment_receipt_%s_%d.txt", currentCustomer,
transactionList[transactionCount-1].transactionId);
FILE *paymentFile = fopen(receiptFilename, "w");
if (paymentFile != NULL) {
    fprintf(paymentFile, "========== PAYMENT RECEIPT ==========\n");
    fprintf(paymentFile, "Transaction ID: %d\n", transactionList[transactionCount-
1].transactionId);
    fprintf(paymentFile, "Customer: %s\n", currentCustomer);
    fprintf(paymentFile, "Payment Method: %s\n", paymentMethod);
    fprintf(paymentFile, "Customer ID: %s\n", transactionList[transactionCount-
```

```
1].customerId);
        fprintf(paymentFile, "Transaction ID: %s\n", transactionList[transactionCount-
1].transactionIdStr);
        fprintf(paymentFile, "Full Name: %s\n", transactionList[transactionCount-
1].customerNameFull);
        fprintf(paymentFile, "Reference: %s\n", transactionList[transactionCount-
1].reference);
        fprintf(paymentFile, "Amount Paid: $%.2f\n", totalAmount);
        fprintf(paymentFile, "Date: %s\n", transactionList[transactionCount-1].date);
        fprintf(paymentFile, "====================================\n");
        fclose(paymentFile);

        char receiptMsg[150];
        sprintf(receiptMsg, "Payment receipt saved as '%s'", receiptFilename);
        printCentered(receiptMsg);
    }
}

// Login functions Of Our System

void adminLogin() {
    char username[30], password[30];
    printCentered("");
    printCentered("Admin Login");
    printCenteredInput("Username: ");
    scanf("%s", username);
    printCenteredInput("Password: ");
    getHiddenPassword(password);

    for (int i = 0; i < adminCount; i++) {
        if (strcmp(username, adminList[i].username) == 0 &&
            strcmp(password, adminList[i].password) == 0) {
            char welcomeMsg[100];
            sprintf(welcomeMsg, "Admin login successful! Welcome, %s", username);
            printCentered(welcomeMsg);
            adminLoggedIn = 1;
            return;
        }
    }
    printCentered("Invalid username or password!");
```

```
    adminLoggedIn = 0;
}

void employeeRegister() {
    if (employeeCount >= MAX_USERS) {
        printCentered("Highest Employee limit reached!");
        return;
    }

    printCentered("");
    printCentered("Employee Register");
    printCenteredInput("Username: ");
    scanf("%s", employeeList[employeeCount].username);
    printCenteredInput("Password: ");
    getHiddenPassword(employeeList[employeeCount].password);
    employeeCount++;
    saveEmployees();
    printCentered("Employee registered successfully and saved to file!");
}

void employeeLogin() {
    char username[30], password[30];
    printCentered("");
    printCentered("Employee Login");
    printCenteredInput("Username: ");
    scanf("%s", username);
    printCenteredInput("Password: ");
    getHiddenPassword(password);

    for (int i = 0; i < employeeCount; i++) {
        if (strcmp(username, employeeList[i].username) == 0 &&
            strcmp(password, employeeList[i].password) == 0) {
            char welcomeMsg[100];
            sprintf(welcomeMsg, "Employee login successful! Welcome, %s", username);
            printCentered(welcomeMsg);
            employeeLoggedIn = 1;
            strcpy(currentEmployee, username);
            return;
        }
    }
```

```
    printCentered("Login failed!");
    employeeLoggedIn = 0;
}

void customerRegister() {
    if (customerCount >= MAX_USERS) {
        printCentered("Highest Customer limit reached!");
        return;
    }

    printCentered("");
    printCentered("Customer Register");
    printCenteredInput("Username: ");
    scanf("%s", customerList[customerCount].username);
    printCenteredInput("Password: ");
    getHiddenPassword(customerList[customerCount].password);
    customerCount++;
    saveCustomers();
    printCentered("Customer registered successfully and saved to file!");
}


void customerLogin() {
    char username[30], password[30];
    printCentered("");
    printCentered("Customer Login");
    printCenteredInput("Username: ");
    scanf("%s", username);
    printCenteredInput("Password: ");
    getHiddenPassword(password);


    for (int i = 0; i < customerCount; i++) {
        if (strcmp(username, customerList[i].username) == 0 &&
            strcmp(password, customerList[i].password) == 0) {
            char welcomeMsg[100];
            sprintf(welcomeMsg, "Customer login successful! Welcome, %s", username);
            printCentered(welcomeMsg);
            customerLoggedIn = 1;
            strcpy(currentCustomer, username);
```

```
            return;
        }
    }
    printCentered("Login failed!");
    customerLoggedIn = 0;
}


// Menu functions of our System

void adminMenu() {
    int choice;

    if (!adminLoggedIn) {
        printCentered("");
        printCentered("---------- Admin Menu ----------");
        printCentered("1. Login");
        printCenteredInput("Choose option: ");
        scanf("%d", &choice);
        if (choice == 1) {
            adminLogin();
        } else {
            printCentered("Invalid option!");
            return;
        }
    }

    while (adminLoggedIn) {
        printCentered("");
        printCentered("---------- Admin Actions ----------");
        printCentered("1. Add Item");
        printCentered("2. View All Items");
        printCentered("3. View Items by Category");
        printCentered("4. Update Item");
        printCentered("5. Remove Item");
        printCentered("6. Save All Data");
        printCentered("7. Logout");
        printCenteredInput("Choose option: ");
        scanf("%d", &choice);
```

```
        switch (choice) {
            case 1: addItem(); break;
            case 2: viewAllItems(); break;
            case 3: viewItemsByCategory(); break;
            case 4: updateItem(); break;
            case 5: removeItem(); break;
            case 6: saveDataToFiles(); break;
            case 7: adminLoggedIn = 0; printCentered("Logged out from admin."); break;
            default: printCentered("Invalid choice!");
        }
    }
}

void employeeMenu() {
    int choice;


    if (!employeeLoggedIn) {
        printCentered("");
        printCentered("---------- Employee Menu ----------");
        printCentered("1. Register");
        printCentered("2. Login");
        printCenteredInput("Choose option: ");
        scanf("%d", &choice);

        if (choice == 1) employeeRegister();
        else if (choice == 2) employeeLogin();
        else printCentered("Invalid option!");
    }

    while (employeeLoggedIn) {
        printCentered("");
        printCentered("---------- Employee Actions ----------");
        printCentered("1. Generate Report");
        printCentered("2. Supply Product");
        printCentered("3. Receive Stock");
        printCentered("4. Receive Payment");
        printCentered("5. View All Items");
        printCentered("6. View Items by Category");
        printCentered("7. Save All Data");
```

```
        printCentered("8. Logout");
        printCenteredInput("Choose option: ");
        scanf("%d", &choice);


        switch (choice) {
            case 1: generateEmployeeReport(); break;
            case 2: supplyProduct(); break;
            case 3: receiveStock(); break;
            case 4: receivePayment(); break;
            case 5: viewAllItems(); break;
            case 6: viewItemsByCategory(); break;
            case 7: saveDataToFiles(); break;
            case 8:
                employeeLoggedIn = 0;
                strcpy(currentEmployee, "");
                printCentered("Logged out from employee account.");
                break;
            default: printCentered("Invalid choice!");
        }
    }
}


void customerMenu() {
    int choice;

    if (!customerLoggedIn) {
        printCentered("");
        printCentered("---------- Customer Menu ----------");
        printCentered("1. Register");
        printCentered("2. Login");
        printCenteredInput("Choose option: ");
        scanf("%d", &choice);

        if (choice == 1) customerRegister();
        else if (choice == 2) customerLogin();
        else printCentered("Invalid option!");
    }
```

```
    while (customerLoggedIn) {
        printCentered("");
        printCentered("---------- Customer Actions ----------");
        printCentered("1. View All Items");
        printCentered("2. View Items by Category");
        printCentered("3. Place Order");
        printCentered("4. Receive Order");
        printCentered("5. Process Payment");
        printCentered("6. Logout");
        printCenteredInput("Choose option: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: viewAllItems(); break;
            case 2: viewItemsByCategory(); break;
            case 3: placeOrder(); break;
            case 4: receiveOrder(); break;
            case 5: processPayment(); break;
            case 6:
                customerLoggedIn = 0;
                strcpy(currentCustomer, "");
                printCentered("Logged out from customer account.");
                break;
            default: printCentered("Invalid choice!");
        }
    }
}

int main() {
    int choice;
    loadDataFromFiles();
    printCentered("System initialized! Data loaded from files.");

    displayFullAsciiLogo();

    while (1) {
        printCentered("");

printCentered("============================================");
```

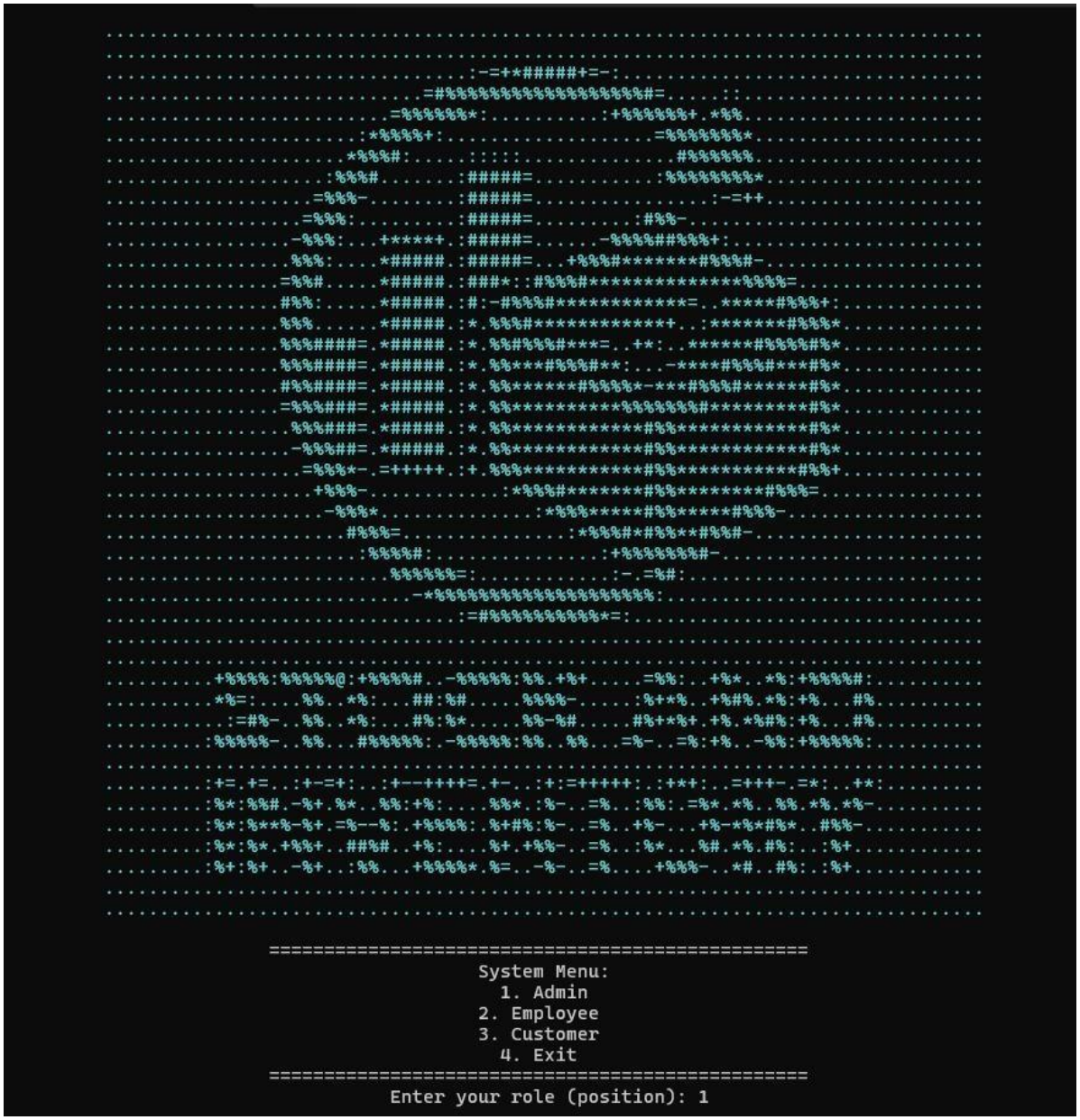
```
        printCentered("System Menu:");
        printCentered("1. Admin");
        printCentered("2. Employee");
        printCentered("3. Customer");
        printCentered("4. Save All Data & Exit");

printCentered("================================================");
        printCenteredInput("Enter your role (position): ");
        if (scanf("%d", &choice) != 1) {
            int c; while ((c = getchar()) != '\n' && c != EOF) {}
            printCentered("Invalid input. Try again.");
            continue;
        }
        switch (choice) {
            case 1: adminMenu(); break;
            case 2: employeeMenu(); break;
            case 3: customerMenu(); break;
            case 4:
                saveDataToFiles();
                printCentered("All data saved successfully!");
                printCentered("Exiting From The System....");
                return 0;
            default: printCentered("Invalid choice! Kindly try again.");
        }
    }
    return 0;
}
```

USER MANUAL

Step 1: Launch Application [System Logo Display]

- The system displays a comprehensive ASCII art logo upon startup
- Data is automatically loaded from existing text files
- System confirms successful initialization with file loading status
- The main logo represents the professional identity of the Stock & Inventory Management System



Step 2: Admin Login Process [Admin Login Screen]

- Navigate to Admin menu by selecting option 1 from main menu
- System prompts for username entry (visible text input)
- Password entry uses secure masking (asterisks replace characters)
- Default admin accounts: Hamja/Hamja123, Sabbir/Sabbir123, Nafisha/Nafisha123
- Successful authentication displays welcome message with username
- Failed authentication returns to main menu with error notification

Step 3: Admin Menu Options [Admin Actions Menu]

- Upon successful login, seven administrative options appear:
    1. Add Item - Create new inventory entries
    2. View All Items - Display complete inventory listing
    3. View Items by Category - Filtered inventory viewing
    4. Update Item - Modify existing inventory records
    5. Remove Item - Delete inventory entries
    6. Save All Data - Manual data persistence trigger
    7. Logout - Secure session termination

Step 4: Adding New Inventory Items [Add Item Interface]

- Select "Add Item" from admin menu
- Enter item name (alphanumeric, spaces allowed)
- Input item price (decimal values accepted)
- Choose from predefined categories:
    - Electronics: Phone, Laptop, Monitor, Electronics
    - Food Items: Fruit, Vegetable, Meat, Drinks
- Enter initial quantity (integer values)
- System confirms successful addition and saves to items.txt file
- New item appears in subsequent inventory listings

Step 5: Viewing Inventory Items [Inventory Display]

- "View All Items" displays complete inventory with:
  - Sequential numbering for easy reference
  - Item name, category, and pricing information
  - Current stock levels with low-stock indicators
  - Items below threshold marked with [LOW STOCK!] warning
- Category-based viewing filters items by selected category
- Empty inventory displays appropriate "No items to display" message

Step 6: Updating Existing Items [Update Item Process]

- Select "Update Item" to modify inventory records
- System displays current inventory for selection
- Enter item number corresponding to desired item
- Current item details displayed for reference
- Modify individual fields:
  - Name: Enter new name or "." to keep current
  - Price: Enter new price or negative value to keep current
  - Category: Select from category list or invalid number to keep current
  - Quantity: Enter new quantity or negative value to keep current
- Changes saved immediately to items.txt file
- Confirmation message indicates successful update

Step 7: Removing Inventory Items [Remove Item Interface]

- Access "Remove Item" from admin menu
- Current inventory displayed with numbered references
- Enter item number for deletion (1-based indexing)
- Invalid numbers trigger error messages
- Successful removal shifts remaining items in array
- Changes immediately saved to items.txt file
- Confirmation message verifies successful deletion

```
================================================
                  System Menu:
                    1. Admin
                  2. Employee
                  3. Customer
                    4. Exit
================================================
          Enter your role (position): 1

         ---------- Admin Menu ----------
                    1. Login
                Choose option: 1

                  Admin Login
                   Username: Hamja
                   Password: ********
      Admin login successful! Welcome, Hamja

        ---------- Admin Actions ----------
                    1. Add Item
                2. View All Items
            3. View Items by Category
                  4. Update Item
                  5. Remove Item
                    6. Logout
```

```
            Choose option: 1

               Add New Item
          Enter item name: Asus 07
          Enter item price: 67000

          Select a category:
               1. Phone
              2. Laptop
              3. Monitor
            4. Electronics
               5. Fruit
             6. Vegetable
               7. Meat
              8. Drinks
      Enter category number: 2
          Enter quantity: 10
      Item added successfully!
```

```
---------- Admin Actions ----------
1. Add Item
2. View All Items
3. View Items by Category
4. Update Item
5. Remove Item
6. Logout
Choose option: 2

---------- List of Items ----------
1. Asus 07 | Category: Laptop | $67000.00 | Stock: 10
2. Banana | Category: Fruit | $10.00 | Stock: 20
3. Mojo | Category: Drinks | $20.00 | Stock: 40
4. Chicken | Category: Meat | $220.00 | Stock: 20
5. Redmi | Category: Phone | $22000.00 | Stock: 30
```

```
---------- Admin Actions ----------
1. Add Item
2. View All Items
3. View Items by Category
4. Update Item
5. Remove Item
6. Logout
Choose option: 3

Select category to view:
1. Phone
2. Laptop
3. Monitor
4. Electronics
5. Fruit
6. Vegetable
7. Meat
8. Drinks
Enter category number: 1

---------- Items in Category: Phone ----------
5. Redmi | $22000.00 | Stock: 30
6. Vivo 11 pro | $13000.00 | Stock: 23
7. Iphone 11 | $70000.00 | Stock: 12
```

```
                    Choose option: 4

           ---------- Update Item ----------

           ---------- List of Items ----------
 1. Asus 07 | Category: Laptop | $67000.00 | Stock: 10
    2. Banana | Category: Fruit | $10.00 | Stock: 20
    3. Mojo | Category: Drinks | $20.00 | Stock: 40
   4. Chicken | Category: Meat | $220.00 | Stock: 20
   5. Redmi | Category: Phone | $22000.00 | Stock: 30
6. Vivo 11 pro | Category: Phone | $13000.00 | Stock: 23
 7. Iphone 11 | Category: Phone | $70000.00 | Stock: 12
    8. Mango | Category: Fruit | $60.00 | Stock: 45
              Enter item number to update: 6
Updating item: Vivo 11 pro ($13000.00, Phone, stock 23)
                    Enter new name: Samsung
                   Enter new price: 34000

                  Select new category:
                        1. Phone
                       2. Laptop
                       3. Monitor
                     4. Electronics
                        5. Fruit
                      6. Vegetable
                        7. Meat
                       8. Drinks
                  Enter category number: 1
                   Enter new quantity : 15
                 Item updated successfully!
```

```
---------- Admin Actions ----------
1. Add Item
2. View All Items
3. View Items by Category
4. Update Item
5. Remove Item
6. Logout
Choose option: 5

---------- Remove Item ----------

---------- List of Items ----------
1. Asus 07 | Category: Laptop | $67000.00 | Stock: 10
2. Banana | Category: Fruit | $10.00 | Stock: 20
3. Mojo | Category: Drinks | $20.00 | Stock: 40
4. Chicken | Category: Meat | $220.00 | Stock: 20
5. Redmi | Category: Phone | $22000.00 | Stock: 30
6. Samsung | Category: Phone | $34000.00 | Stock: 15
7. Iphone 11 | Category: Phone | $70000.00 | Stock: 12
8. Mango | Category: Fruit | $60.00 | Stock: 45
Enter the item number to remove: 5
Item removed successfully.
```

```
---------- Admin Actions ----------
1. Add Item
2. View All Items
3. View Items by Category
4. Update Item
5. Remove Item
6. Logout
Choose option: 6
Logged out from admin.
```

Step 8: Employee Registration [Employee Registration]

- New employees select "Register" from employee menu
- Username entry (unique identifier required)
- Password entry with secure masking display
- System checks against maximum user limits
- Successful registration saves to employees.txt file
- Registration confirmation enables immediate login capability

Employee Authentication

Step 9: Employee Login Process [Employee Login]

- Existing employees select "Login" option
- Username and password entry with secure masking
- Credentials verified against employees.txt file
- Successful login displays personalized welcome message
- Current employee name stored for session tracking
- Failed login returns to employee menu with error notification

Employee Dashboard

Step 10: Employee Operations Menu [Employee Actions Menu]

- Authenticated employees access eight operational functions:
    1. Generate Report - Comprehensive system reporting
    2. Supply Product - Stock replenishment operations
    3. Receive Stock - Stock level monitoring and alerts
    4. Receive Payment - Transaction processing oversight
    5. View All Items - Inventory viewing capabilities
    6. View Items by Category - Filtered inventory access
    7. Save All Data - Manual data persistence
    8. Logout - Secure session termination

Stock Replenishment Process

Step 11: Supplying Products [Supply Product Interface]

- Select "Supply Product" for stock replenishment
- Current inventory displayed for supplier reference
- Enter item number requiring stock increase

- Input supply quantity (positive integers only)
- Enter supplier name for tracking purposes
- System automatically:
    - Increases item stock levels
    - Records supply transaction with date
    - Updates supply_records.txt file
    - Displays new stock level confirmation

Stock Alert Management

Step 12: Monitoring Stock Levels [Stock Alert Display]

- "Receive Stock" displays low-stock alert report
- Items below threshold (5 units) automatically identified
- Alert format includes:
    - Item name with ALERT designation
    - Current stock level
    - Product category information
- Empty alert list indicates sufficient stock levels
- Total count of items requiring restock provided
- Critical for proactive inventory management

Employee Reporting System

Step 13: Generate Comprehensive Reports [Employee Report Generation]

- "Generate Report" creates detailed system analysis
- Report includes:
    - Header with employee name and generation date
    - Inventory summary with total items and valuation
    - Low stock alerts section
    - Recent supply records tracking
    - Transaction summary with revenue calculations
- Report automatically saved as employee_report.txt
- Console display provides immediate viewing
- File format enables sharing and archival purposes

```
================================================
                System Menu:
                  1. Admin
                2. Employee
                3. Customer
                  4. Exit
================================================
        Enter your role (position): 2

      ---------- Employee Menu ----------
                  1. Register
                    2. Login
                Choose option: 2

                 Employee Login
                   Username: Ratul
                   Password: ***
  Employee login successful! Welcome, Ratul
```

```
================================================
                System Menu:
                  1. Admin
                2. Employee
                3. Customer
                  4. Exit
================================================
        Enter your role (position): 2

      ---------- Employee Menu ----------
                  1. Register
                    2. Login
                Choose option: 1

               Employee Register
                   Username: Ratul
                   Password: ***
        Employee registered successfully!
```


---------- Employee Actions ----------
1. Generate Report
2. Supply Product
3. Receive Stock
4. Receive Payment
5. View All Items
6. View Items by Category
7. Logout
Choose option: 1
================== EMPLOYEE REPORT ==================
Generated by: Ratul
Date: 16/08/2025
---------- INVENTORY SUMMARY ----------
Total Items: 7
Total Inventory Value: $1894100.00
---------- LOW STOCK ALERTS ----------
No items with low stock.
---------- RECENT SUPPLY RECORDS ----------
No supply records available.
---------- TRANSACTION SUMMARY ----------
Total Transactions: 1
Total Revenue: $134000.00
==

```
---------- Employee Actions ----------
1. Generate Report
2. Supply Product
3. Receive Stock
4. Receive Payment
5. View All Items
6. View Items by Category
7. Logout
Choose option: 2

---------- Supply Product ----------

---------- List of Items ----------
1. Asus 07 | Category: Laptop | $67000.00 | Stock: 8
2. Banana | Category: Fruit | $10.00 | Stock: 20
3. Mojo | Category: Drinks | $20.00 | Stock: 40
4. Chicken | Category: Meat | $220.00 | Stock: 20
5. Samsung | Category: Phone | $34000.00 | Stock: 15
6. Iphone 11 | Category: Phone | $70000.00 | Stock: 12
7. Mango | Category: Fruit | $60.00 | Stock: 45
Enter item number to supply: 4
Enter quantity to supply: 40
Enter supplier name: Faruk
Product supplied successfully! New stock: 60
```

```
---------- Employee Actions ----------
1. Generate Report
2. Supply Product
3. Receive Stock
4. Receive Payment
5. View All Items
6. View Items by Category
7. Logout
Choose option: 3

---------- Stock Alert Report ----------
Items with low stock :

All items have sufficient stock!
```

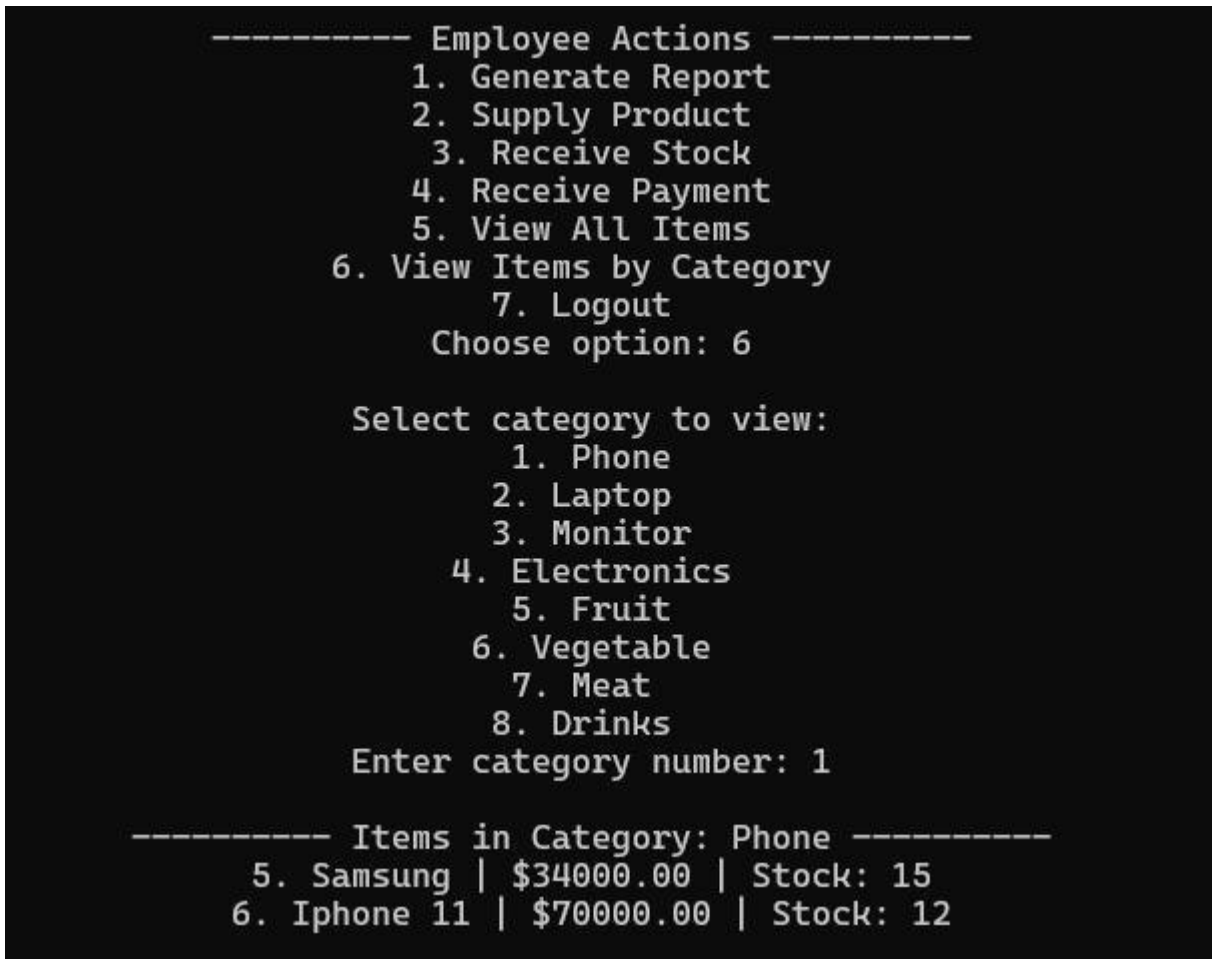
```
---------- Employee Actions ----------
          1. Generate Report
          2. Supply Product
           3. Receive Stock
          4. Receive Payment
          5. View All Items
       6. View Items by Category
             7. Logout
           Choose option: 4

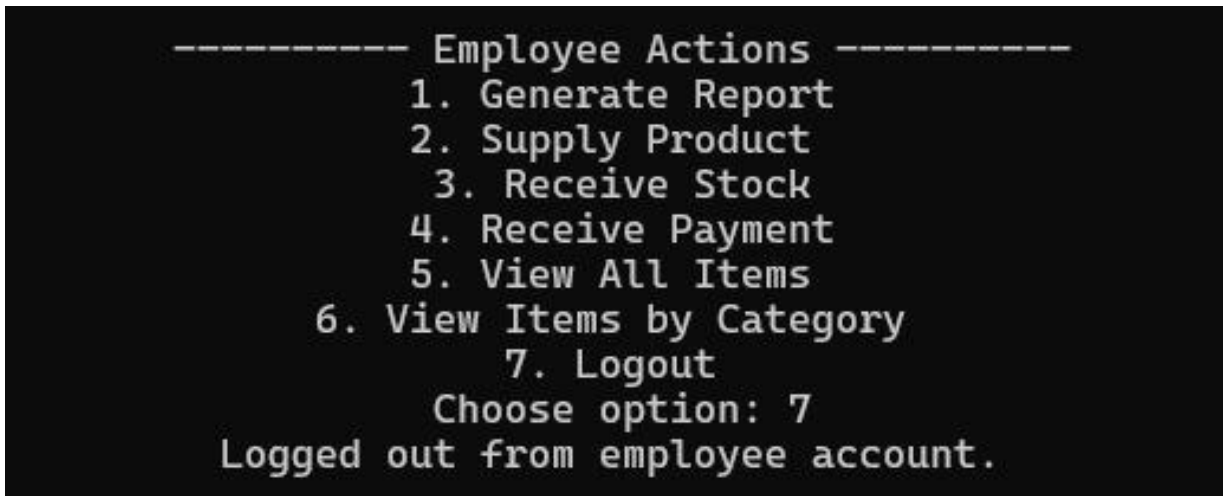
---------- Pending Payments ----------
1. Customer: Jarif | Amount: $134000.00 | Method: Bkash | Date: 16/08/2025

All payments have been processed and recorded.
```

```
---------- Employee Actions ----------
          1. Generate Report
          2. Supply Product
           3. Receive Stock
          4. Receive Payment
          5. View All Items
       6. View Items by Category
             7. Logout
           Choose option: 5

---------- List of Items ----------
1. Asus 07 | Category: Laptop | $67000.00 | Stock: 8
  2. Banana | Category: Fruit | $10.00 | Stock: 20
  3. Mojo | Category: Drinks | $20.00 | Stock: 40
 4. Chicken | Category: Meat | $220.00 | Stock: 60
5. Samsung | Category: Phone | $34000.00 | Stock: 15
6. Iphone 11 | Category: Phone | $70000.00 | Stock: 12
  7. Mango | Category: Fruit | $60.00 | Stock: 45
```


---------- Employee Actions ----------
1. Generate Report
2. Supply Product
3. Receive Stock
4. Receive Payment
5. View All Items
6. View Items by Category
7. Logout
Choose option: 6
Select category to view:
1. Phone
2. Laptop
3. Monitor
4. Electronics
5. Fruit
6. Vegetable
7. Meat
8. Drinks
Enter category number: 1
---------- Items in Category: Phone ----------
5. Samsung | $34000.00 | Stock: 15
6. Iphone 11 | $70000.00 | Stock: 12


---------- Employee Actions ----------
1. Generate Report
2. Supply Product
3. Receive Stock
4. Receive Payment
5. View All Items
6. View Items by Category
7. Logout
Choose option: 7
Logged out from employee account.

Step 14: Customer Registration Process [Customer Registration]

- New customers select "Register" from customer menu
- Username creation (unique identifier required)
- Password setup with secure masking
- System enforces maximum customer limits
- Successful registration saves to customers.txt file
- Immediate login capability upon registration completion

Customer Authentication

Step 15: Customer Login Verification [Customer Login]

- Returning customers select "Login" option
- Username and password verification process
- Credentials checked against customers.txt file
- Successful authentication displays welcome message
- Customer session established for order tracking
- Login failures return to customer menu with errors

Customer Shopping Interface

Step 16: Customer Dashboard Access [Customer Actions Menu]

- Authenticated customers access six primary functions:
    1. View All Items - Browse complete product catalog
    2. View Items by Category - Filtered product browsing
    3. Place Order - Initiate purchase process
    4. Receive Order - Order status management
    5. Process Payment - Complete transaction processing
    6. Logout - Secure session termination

Product Browsing Experience

Step 17: Viewing Available Products [Customer Inventory View]

- Product catalog displays comprehensive item information:
    - Item names and categories
    - Current pricing information
    - Available stock quantities
    - Low stock warnings for limited availability

- Category-based browsing enables focused shopping
- Stock levels help customers make informed decisions
- Real-time inventory ensures accurate availability

Order Placement Process

Step 18: Creating Customer Orders [Place Order Interface]

- Select "Place Order" to begin purchase process
- Complete inventory displayed for selection
- Enter item number for desired product
- Selected item details displayed for confirmation:
    - Product name and pricing
    - Available stock quantity
- Enter desired purchase quantity
- System validates:
    - Quantity availability against stock
    - Positive quantity requirements
- Order details summary includes:
    - Unique order ID assignment
    - Customer name association
    - Item selection and quantity
    - Total amount calculation
    - Order status (initially "Pending")
    - Order date timestamp
- Order saved to orders.txt file
- Individual order receipt file generated

Order Management System

Step 19: Managing Customer Orders [Order Status Management]

- "Receive Order" displays customer-specific order history
- Order information includes:
    - Order ID for reference
    - Item name and quantity
    - Total amount and status
    - Order date for tracking
- Status update capability:
    - Select order ID for status change
    - Update from "Pending" to "Received"

    - Prevent duplicate status changes
- Order modifications saved to orders.txt file
- Status tracking enables order lifecycle management

Payment Processing System

Step 20: Complete Transaction Processing [Payment Processing]

- "Process Payment" displays pending orders for payment
- Total amount calculation for all pending orders
- Payment method selection:
    1. Visa - Credit/debit card processing
    2. PayPal - Online payment platform
    3. Bkash - Mobile financial service
    4. Nagad - Digital payment system
- Required payment information:
    - Customer ID/Number entry
    - Transaction ID input
    - Full name confirmation
    - Reference information (optional)
- Payment confirmation includes:
    - Transaction ID assignment
    - Payment method confirmation
    - Total amount verification
    - Transaction date recording
- Automatic order status update to "Completed"
- Stock levels automatically reduced
- Payment receipt file generated
- Transaction saved to transactions.txt file

System Administration Features

Data Management Operations

Step 21: Data Persistence and Backup [Data Saving Operations]

- "Save All Data" option available in all user menus
- Comprehensive data writing to multiple files:
    - items.txt - Inventory data
    - orders.txt - Order information
    - transactions.txt - Payment records

    - supply_records.txt - Supply history
    - employees.txt - Employee accounts
    - customers.txt - Customer accounts
    - admins.txt - Administrator accounts
- Confirmation messages for each file operation
- Backup system creates timestamped folders
- Data validation ensures file integrity

System Shutdown Process

Step 22: Secure System Exit [System Exit Confirmation]

- Select "Save All Data & Exit" from main menu
- Automatic data saving to all relevant files
- System confirmation of successful data persistence
- Graceful application termination
- All user sessions properly closed
- File handles properly released

Troubleshooting and Error Handling

Common Error Scenarios

Step 23: Error Management

- Invalid menu selections display error messages and menu redisplay
- Authentication failures return to appropriate login screens
- Insufficient stock orders prevent invalid transactions
- File operation failures display error notifications
- Input validation prevents system crashes
- Maximum limit enforcement prevents array overflow

Step 24: Data File Management

- Missing data files trigger warning messages
- System continues operation with empty data structures
- File corruption handled with error messages
- Backup files enable data recovery
- Manual file editing supported through documented formats
- System restart reloads all data files automatically

```
==============================================
                System Menu:
                  1. Admin
                2. Employee
                3. Customer
                  4. Exit
==============================================
        Enter your role (position): 3

        ---------- Customer Menu ----------
                  1. Register
                    2. Login
                Choose option: 1

              Customer Register
                  Username: Jarif
                  Password: ***
       Customer registered successfully!
```

```
              Customer Login
                Username: Jarif
                Password: ***
Customer login successful! Welcome, Jarif

  ---------- Customer Actions ----------
          1. View All Items
      2. View Items by Category
          3. Place Order
          4. Receive Order
          5. Process Payment
              6. Logout
          Choose option: |
```

```
---------- Customer Actions ----------
1. View All Items
2. View Items by Category
3. Place Order
4. Receive Order
5. Process Payment
6. Logout
Choose option: 1

---------- List of Items ----------
1. Asus 07 | Category: Laptop | $67000.00 | Stock: 10
2. Banana | Category: Fruit | $10.00 | Stock: 20
3. Mojo | Category: Drinks | $20.00 | Stock: 40
4. Chicken | Category: Meat | $220.00 | Stock: 20
5. Samsung | Category: Phone | $34000.00 | Stock: 15
6. Iphone 11 | Category: Phone | $70000.00 | Stock: 12
7. Mango | Category: Fruit | $60.00 | Stock: 45
```

```
---------- Customer Actions ----------
1. View All Items
2. View Items by Category
3. Place Order
4. Receive Order
5. Process Payment
6. Logout
Choose option: 2

Select category to view:
1. Phone
2. Laptop
3. Monitor
4. Electronics
5. Fruit
6. Vegetable
7. Meat
8. Drinks
Enter category number: 5

---------- Items in Category: Fruit ----------
2. Banana | $10.00 | Stock: 20
7. Mango | $60.00 | Stock: 45
```

```
---------- Customer Actions ----------
          1. View All Items
       2. View Items by Category
             3. Place Order
            4. Receive Order
           5. Process Payment
               6. Logout
            Choose option: 3

---------- Place Order ----------

---------- List of Items ----------
1. Asus 07 | Category: Laptop | $67000.00 | Stock: 10
   2. Banana | Category: Fruit | $10.00 | Stock: 20
   3. Mojo | Category: Drinks | $20.00 | Stock: 40
  4. Chicken | Category: Meat | $220.00 | Stock: 20
 5. Samsung | Category: Phone | $34000.00 | Stock: 15
6. Iphone 11 | Category: Phone | $70000.00 | Stock: 12
   7. Mango | Category: Fruit | $60.00 | Stock: 45
          Enter item number to order: 1
 Selected: Asus 07 | Price: $67000.00 | Available: 10
          Enter quantity to order: 2

---------- Order Details ----------
            Order ID: 1
          Customer: Jarif
           Item: Asus 07
            Quantity: 2
        Total Amount: $134000.00
           Status: Pending
           Date: 16/08/2025

        Order placed successfully!
```

```
---------- Customer Actions ----------
          1. View All Items
       2. View Items by Category
             3. Place Order
            4. Receive Order
           5. Process Payment
               6. Logout
            Choose option: 4

---------- Your Orders ----------
Order ID: 1 | Item: Asus 07 | Qty: 2 | Amount: $134000.00 | Status: Pending | Date: 16/08/2025
          Enter Order ID to receive : 1
          Order 1 marked as received!
```

```
---------- Customer Actions ----------
1. View All Items
2. View Items by Category
3. Place Order
4. Receive Order
5. Process Payment
6. Logout
Choose option: 5

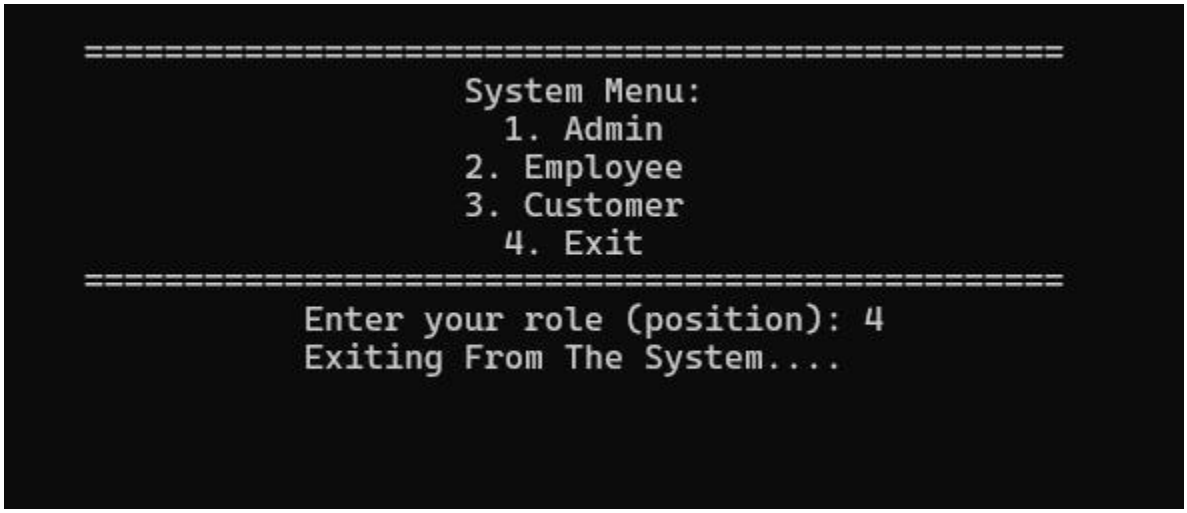
---------- Pending Orders for Payment ----------
Order ID: 1 | Item: Asus 07 | Amount: $134000.00

Total Amount to Pay: $134000.00

---------- Select Payment Method ----------
1. Visa
2. PayPal
3. Bkash
4. Nagad
Choose payment method: 3
Enter Customer ID/Number: 01903787407
Enter Transaction ID: 2fbhe28hdh
Enter Full Name: Md Jarif Ahmed
Enter Reference (optional): Laptop

---------- Payment Confirmation ----------
Transaction ID: 1
Payment Method: Bkash
Amount Paid: $134000.00
Date: 16/08/2025
Payment successful! All orders completed.
```

```
---------- Customer Actions ----------
1. View All Items
2. View Items by Category
3. Place Order
4. Receive Order
5. Process Payment
6. Logout
Choose option: 6
Logged out from customer account.
```



FUTURE SCOPE

1. Database Integration and Scalability

Relational Database Implementation: The most critical enhancement would involve migrating from file-based storage to a robust relational database management system (MySQL, PostgreSQL, or SQLite). This transformation would provide ACID compliance, concurrent user support, and enhanced data integrity. Implementation of proper database schemas with foreign key relationships would eliminate data redundancy and improve query performance.

Cloud Database Integration: Integration with cloud database services (Amazon RDS, Google Cloud SQL, or Azure Database) would provide scalability, automated backups, and global accessibility. This enhancement would support business growth and geographic distribution.

Data Analytics and Business Intelligence: Implementation of advanced analytics capabilities including sales trend analysis, inventory turnover calculations, seasonal demand forecasting, and supplier performance metrics. Integration with analytics platforms could provide predictive insights for inventory optimization.

2. Modern User Interface Development

Web-Based Interface: Development of a responsive web application using modern frameworks (React, Angular, or Vue.js) would provide cross-platform accessibility and

improved user experience. RESTful API development would separate backend logic from frontend presentation, enabling multiple interface options.

Mobile Application Development: Native mobile applications for iOS and Android would enable on-the-go inventory management, barcode scanning capabilities, and real-time notifications. Mobile interfaces would be particularly valuable for warehouse operations and field sales teams.

Desktop GUI Application: Development of modern desktop applications using frameworks like Electron or Qt would provide rich user interfaces with charts, graphs, and advanced data visualization capabilities while maintaining offline functionality.

3. Advanced Inventory Management Features

Barcode and QR Code Integration: Implementation of barcode scanning capabilities would automate product identification, reduce manual entry errors, and accelerate inventory operations. Integration with barcode generation for product labeling would complete the automation cycle.

RFID Technology Integration: Radio Frequency Identification (RFID) implementation would enable real-time inventory tracking, automated stock counts, and enhanced theft prevention. RFID integration would particularly benefit large-scale warehouse operations.

Multi-Location Inventory Management: Support for multiple warehouses, stores, or distribution centers with inter-location transfer capabilities would accommodate business expansion and supply chain optimization.

Advanced Categorization: Implementation of hierarchical product categories, custom attributes, and product variants (size, color, model) would provide more sophisticated inventory organization and search capabilities.

4. Enhanced Business Process Automation

Automated Reordering System: Implementation of intelligent reordering based on sales velocity, seasonal trends, and supplier lead times would optimize inventory levels and reduce stockouts. Integration with supplier systems could automate the entire procurement process.

Supplier Portal Integration: Development of supplier interfaces enabling direct catalog updates, order acknowledgments, and shipment tracking would streamline supply chain operations and improve communication.

Customer Self-Service Portal: Implementation of customer portals for order tracking, return processing, and account management would reduce administrative overhead while improving customer satisfaction.

Workflow Management: Advanced approval workflows for purchases, returns, and inventory adjustments would provide better control and audit trails for business operations.

5. Integration and Connectivity Enhancements

ERP System Integration: Integration with Enterprise Resource Planning systems (SAP, Oracle, or Microsoft Dynamics) would provide seamless data flow between inventory management and other business processes including accounting, human resources, and customer relationship management.

E-commerce Platform Integration: Direct integration with e-commerce platforms (Shopify, WooCommerce, or Magento) would synchronize online and offline inventory, preventing overselling and providing real-time product availability.

Payment Gateway Integration: Implementation of secure payment gateways (Stripe, PayPal, or Square) would enable real-time payment processing, fraud detection, and automated reconciliation.

API Development: Comprehensive RESTful API development would enable third-party integrations, custom extensions, and ecosystem development around the core inventory management platform.

6. Security and Compliance Enhancements

Advanced Security Implementation: Multi-factor authentication, role-based permissions with granular controls, and audit logging would provide enterprise-level security. Implementation of encryption for data at rest and in transit would ensure data protection.

Compliance Management: Support for regulatory compliance including tax calculations, financial reporting standards, and industry-specific requirements (FDA for food products, FCC for electronics) would broaden the system's applicability.

Blockchain Integration: Blockchain implementation for supply chain traceability would provide immutable records of product journey from manufacturer to customer, particularly valuable for high-value or regulated products.

7. Advanced Analytics and Reporting

Machine Learning Integration: Implementation of machine learning algorithms for demand forecasting, anomaly detection, and optimization recommendations would provide intelligent insights for business decision-making.

Real-time Dashboard Development: Interactive dashboards with real-time metrics, key performance indicators, and customizable widgets would provide executives and managers with immediate business insights.

Advanced Reporting Engine: Flexible report generation with custom filters, scheduled reports, and export capabilities (PDF, Excel, CSV) would support various business reporting requirements.

8. Operational Enhancements

Multi-Currency Support: Implementation of multi-currency capabilities with real-time exchange rates would enable international operations and global supply chain management. Currency conversion automation would simplify international transactions and financial reporting.

Advanced Inventory Valuation: Implementation of multiple inventory valuation methods (FIFO, LIFO, Weighted Average) would provide flexibility for accounting requirements and tax optimization. Support for landed cost calculations including shipping, duties, and handling fees would improve cost accuracy.

Lot and Serial Number Tracking: Individual item tracking through lot numbers or serial numbers would enable precise traceability, warranty management, and recall procedures. This feature would be particularly valuable for electronics, pharmaceuticals, and automotive parts.

Expiration Date Management: Implementation of expiration date tracking with automated alerts for perishable goods would reduce waste and ensure product quality. FEFO (First Expired, First Out) rotation management would optimize inventory turnover.

9. Communication and Notification Systems

Real-time Notification System: Implementation of real-time notifications through email, SMS, and push notifications would keep stakeholders informed of critical events including low stock alerts, order completions, and system updates.

Communication Hub: Internal messaging system enabling communication between different user roles would facilitate coordination and problem resolution. Integration with external communication platforms (Slack, Microsoft Teams) would enhance collaboration.

Customer Communication Automation: Automated customer notifications for order confirmations, shipping updates, and delivery confirmations would improve customer experience while reducing manual administrative work.

10. Quality Assurance and Testing Framework

Automated Testing Suite: Implementation of comprehensive unit tests, integration tests, and end-to-end testing would ensure system reliability and facilitate continuous development. Test-driven development practices would improve code quality and reduce bugs.

Performance Monitoring: Real-time performance monitoring with alerts for system issues would ensure optimal system performance and user experience. Implementation of logging and monitoring tools would facilitate troubleshooting and system optimization.

Data Backup and Recovery: Automated backup systems with point-in-time recovery capabilities would protect against data loss. Implementation of disaster recovery procedures would ensure business continuity.

11. Customization and Configuration

Configurable Business Rules: Flexible configuration options for business rules including pricing strategies, discount structures, and workflow approvals would accommodate different business models and requirements.

Custom Field Support: User-defined custom fields for products, customers, and orders would accommodate specific business requirements without requiring code modifications.

Theme and Branding Customization: Customizable user interface themes and branding options would enable organizations to maintain their visual identity across the system.

12. Advanced Features for Specific Industries

Manufacturing Integration: Support for bill of materials (BOM), production planning, and work order management would extend the system's applicability to manufacturing environments.

Retail-Specific Features: Point-of-sale integration, customer loyalty programs, and promotion management would make the system suitable for retail operations.

Healthcare Compliance: Integration with healthcare regulations (HIPAA, FDA) and support for medical device tracking would enable use in healthcare environments.

Food Service Features: Recipe management, nutritional information tracking, and food safety compliance would support restaurant and food service operations.

IMPLEMENTATION ROADMAP

Phase 1 (2 months): Database migration, basic web interface, and enhanced security features.
Phase 2 (3 months): Mobile applications, advanced reporting, and API development.
Phase 3 (4 months): Machine learning integration, advanced analytics, and ERP integrations.
Phase 4 (5 months): Blockchain integration, IoT connectivity, and industry-specific features.

TECH RECOMMENDATIONS

Backend Development: Node.js with Express.js or Python with Django/Flask for scalable server-side development.
Database Systems: PostgreSQL / MySQL for relational data, MongoDB for document storage.
Frontend Development: React.js or Vue.js for web interfaces, React Native or Flutter for mobile applications.
Cloud Infrastructure: AWS, Google Cloud Platform, or Microsoft Azure for scalable hosting
Analytics Platform: Apache Spark or Elasticsearch for big data processing and analytics.

REFERENCE

The proper guidelines from Our instructor Md. Sakib Ali Mazumder sir.

CONCLUSION

The Stock and Inventory Management System provides an effective and streamlined solution for tracking and managing products, transactions, and customer data. With secure transaction handling, efficient database integration, and a user-friendly interface, it ensures smooth operations even under high loads. The system supports accurate stock monitoring, reduces manual errors, and enables admins to make informed decisions through real-time data access. This platform forms a solid base for scaling into a more advanced inventory ecosystem with features like analytics, barcode integration, and automated reporting in the future.

This project serves as an excellent example of how fundamental programming concepts can be applied to create valuable business solutions, while also highlighting the continuous evolution possible in software development. The extensive future scope ensures that the project remains relevant and extensible as technology and business requirements continue to evolve.